

4 METODOLOGIA

Este capítulo apresenta os procedimentos metodológicos adotados para o desenvolvimento da pesquisa, com o objetivo de descrever de forma sistemática os caminhos utilizados para avaliar experimentalmente o uso de um agente baseado em “*Large Language Model*” (LLM) na tomada de decisão para orquestração de tarefas assíncronas em uma arquitetura simulada de microserviços. De acordo com Antonio GIL (2022), os procedimentos metodológicos correspondem aos métodos e técnicas utilizados para alcançar os objetivos de um estudo de forma organizada e fundamentada. Nesse sentido, a presente pesquisa estrutura-se a partir de uma abordagem quali-quantitativa, com predominância quantitativa, tipologia exploratória, descritiva, explicativa e experimental, utilizando como base a implementação em “*Python*” de um ambiente controlado utilizando “*Docker Compose*”, conforme Newman (2015).

A pesquisa parte da construção de um ambiente experimental controlado, implementado com “*Python*” e “*Docker Compose*”, no qual serão comparadas duas abordagens de orquestração: uma tradicional, baseada em regras determinísticas, e outra baseada em agente LLM. A escolha por microserviços como contexto arquitetural é fundamentada em Fowler e Lewis (2017), que descrevem microserviços como serviços organizados em torno de capacidades de negócio, executados em processos próprios e comunicando-se por mecanismos leves, além de serem independentemente implantáveis. Também se apoia em Newman (2015), que destaca benefícios como limites modulares fortes, implantação independente e diversidade tecnológica em arquiteturas de microserviços.

A utilização de tarefas assíncronas se justifica pela própria natureza distribuída dos microserviços. Richardson, ao tratar de padrões para microserviços, apresenta problemas recorrentes de decomposição, comunicação, transações distribuídas e coordenação entre serviços, incluindo o padrão *Saga*, no qual transações locais publicam mensagens ou eventos para acionar as próximas etapas do processo. Dessa forma, o ambiente experimental proposto busca representar, em escala reduzida, desafios reais de coordenação distribuída, falhas parciais, reexecução e adaptação de fluxo.

4.1 CARACTERIZAÇÃO DA PESQUISA

Quanto aos objetivos, esta pesquisa caracteriza-se como exploratória, descritiva e explicativa. A classificação exploratória justifica-se porque o uso de agentes baseados em LLM para tomada de decisão em orquestração assíncrona de microserviços ainda é um tema em consolidação, exigindo maior familiaridade com o problema, levantamento bibliográfico e análise de exemplos relacionados. Conforme Antonio GIL (2022), pesquisas exploratórias são adequadas para temas pouco explorados, pois auxiliam na compreensão inicial do fenômeno e na formulação

de hipóteses ou caminhos de investigação.

A pesquisa também é descritiva, pois pretende registrar, classificar e comparar o comportamento de duas arquiteturas simuladas: uma com orquestração tradicional baseada em regras e outra com orquestração baseada em agente LLM. Essa classificação está alinhada à definição de pesquisa descritiva apresentada por Antonio GIL (2022), segundo a qual esse tipo de estudo busca descrever características, comportamentos ou relações entre variáveis, sem necessariamente partir de uma explicação causal definitiva.

A dimensão explicativa aparece na comparação entre as abordagens, pois a pesquisa buscará verificar se a introdução do agente LLM influencia os resultados obtidos em relação à orquestração tradicional. O material metodológico anexado indica que a pesquisa explicativa busca identificar fatores que determinam ou influenciam fenômenos, assumindo frequentemente a forma de pesquisa experimental.

Quanto aos procedimentos técnicos, o trabalho também se caracteriza como pesquisa experimental, pois será definido um objeto de estudo, serão selecionadas variáveis, serão controladas condições de execução e serão observados os efeitos produzidos pelas diferentes formas de orquestração. Esse enquadramento segue a definição apresentada no resumo metodológico, segundo a qual a pesquisa experimental determina um objeto de estudo, seleciona variáveis que o influenciam e define formas de controle e observação dos efeitos produzidos .

4.1.1 Abordagem da Pesquisa

A pesquisa adota uma abordagem quali-quantitativa, com predominância quantitativa. A dimensão quantitativa está relacionada à coleta e análise de métricas objetivas obtidas durante os experimentos, incluindo latência média, latência P95, “*throughput*”, taxa de falhas, tempo de recuperação, uso de CPU, uso de memória, tamanho das filas, tempo de decisão do orquestrador e, na abordagem baseada em LLM, tempo de inferência do modelo. Essas métricas permitirão comparar o comportamento das duas abordagens de orquestração com base em indicadores mensuráveis de desempenho, eficiência, resiliência e custo computacional.

Conforme Creswell e Creswell (2021) e Antonio GIL (2022), a abordagem quantitativa é adequada quando os dados são coletados de forma objetiva e sistemática, permitindo análise estatística, identificação de padrões e comparação entre variáveis.

A dimensão qualitativa será utilizada de forma complementar, especialmente na interpretação dos comportamentos observados, das limitações da abordagem baseada em LLM e dos riscos arquiteturais identificados. De acordo com Creswell e Creswell (2021), a abordagem qualitativa permite compreender fenômenos de forma mais aprofundada e interpretativa, buscando identificar padrões e significados subjacentes.

Dessa forma, a escolha por uma abordagem quali-quantitativa justifica-se porque o estudo combina análise estatística de métricas experimentais com interpretação técnica dos resultados.

Essa combinação permite avaliar não apenas se há diferença entre as arquiteturas, mas também em quais cenários a abordagem baseada em LLM apresenta vantagens, limitações ou riscos.

4.1.2 Sujeitos de Pesquisa / Amostra

Este estudo não terá pessoas como sujeitos de pesquisa. O alvo da investigação será composto por artefatos computacionais, ou seja, os componentes implementados no ambiente experimental.

Os objetos de análise serão a arquitetura simulada de microserviços, o orquestrador tradicional baseado em regras, o agente orquestrador baseado em LLM, os registros de execução e as métricas coletadas durante os experimentos. A unidade de análise será cada execução experimental realizada sob um cenário específico. Cada combinação entre cenário e abordagem de orquestração será executada um número previamente definido de repetições, representado por N_{rep} , mantido constante entre as abordagens comparadas. O valor final de N_{rep} será estabelecido antes do início da coleta definitiva, considerando o tempo disponível para execução, a estabilidade das métricas e a necessidade de produzir amostras suficientes para a análise estatística prevista. Dessa forma, a repetição experimental não será definida após a observação dos resultados, reduzindo o risco de seleção oportunista do número de execuções.

4.2 USO DE INTELIGÊNCIA ARTIFICIAL GENERATIVA COMO FERRAMENTA DE APOIO

Em conformidade com a Portaria CNPq nº 2.664/2026 CNPq (2026), que estabelece diretrizes de integridade na atividade científica, este trabalho declara o uso de ferramentas de Inteligência Artificial Generativa como apoio ao desenvolvimento da pesquisa. A utilização dessas ferramentas ocorreu de forma auxiliar, sem substituição da autoria humana, da leitura crítica das referências, da definição metodológica, da análise dos resultados ou da responsabilidade intelectual pelo conteúdo final.

Foram utilizadas as seguintes ferramentas de apoio: o GPT-5.5 Thinking, da OpenAI, para apoio na organização de ideias, revisão textual, estruturação de seções, esclarecimento conceitual e auxílio na redação preliminar de trechos posteriormente revisados pelo autor; o Consensus, para apoio à busca acadêmica, identificação inicial de referências e verificação de aderência temática de artigos relacionados a microserviços, orquestração, agentes baseados em “*Large Language Model*” e sistemas distribuídos; e o Nano Banana Pro, para apoio à geração de imagens e diagramas conceituais utilizados na representação visual dos conceitos discutidos no trabalho.

O uso dessas ferramentas teve caráter instrumental e complementar. As ferramentas de IA não foram utilizadas para fabricar dados, gerar resultados experimentais, substituir a leitura das obras originais, criar referências inexistentes ou realizar a análise final dos dados. As referências

selecionadas foram conferidas e validadas pelo autor, e os conteúdos gerados foram revisados, ajustados e integrados ao texto sob responsabilidade humana.

Dessa forma, o autor declara assumir integral responsabilidade pelo conteúdo final deste trabalho, incluindo a seleção das fontes, a interpretação da literatura, a definição dos procedimentos metodológicos, a execução experimental, a análise dos dados e eventuais imprecisões decorrentes do uso de ferramentas de Inteligência Artificial Generativa.

4.3 ARQUITETURA SIMULADA

O contexto da pesquisa será um ambiente simulado de microserviços, executado localmente por meio de “*Docker Compose*”. A utilização dessa tecnologia permitirá criar contêineres independentes para cada microserviço, reproduzindo uma arquitetura distribuída em escala reduzida. A escolha é coerente com a proposta de microserviços, pois permite representar serviços isolados, com implantação individual e comunicação entre componentes.

O ambiente experimental será implementado em “*Python*”, linguagem escolhida por sua aplicabilidade em prototipação, automação, integração com APIs e desenvolvimento de serviços web. O uso de “*Docker Compose*” possibilitará padronizar a execução dos serviços, reduzir interferências externas e facilitar a repetição dos experimentos.

A implementação dos microserviços será realizada em “*Python*”, utilizando “*FastAPI*” para a interface HTTP do Serviço de Pedidos. A execução das tarefas assíncronas será apoiada pelo “*Celery*”, tendo o “*RabbitMQ*” como “*broker*” de mensagens. Nesse arranjo, o “*Celery*” será utilizado como mecanismo de despacho e consumo de tarefas, enquanto a escolha das ações de orquestração permanecerá sob responsabilidade do mecanismo decisório avaliado. Dessa forma, políticas como RETRY, WAIT, FALLBACK e ABORT não serão delegadas automaticamente ao *framework* de tarefas, preservando a variável experimental concentrada no “*RulesDecisionEngine*” ou no “*LLMDecisionEngine*”. Os recursos automáticos de *autoretry* do “*Celery*” também não serão utilizados para substituir o RETRY decidido pelo orquestrador. A abordagem baseada em LLM utilizará “*runtime*” local por meio do “*Ollama*”.

Assim, cabe distinguir os papéis dos componentes de infraestrutura e do mecanismo decisório: o “*Celery*” transporta e executa tarefas, o “*RabbitMQ*” atua como “*broker*” de mensagens e nenhum dos dois seleciona a ação experimental; a seleção da ação é responsabilidade exclusiva do mecanismo de decisão avaliado (“*RulesDecisionEngine*” ou “*LLMDecisionEngine*”).

Para garantir comparabilidade entre as abordagens, o ambiente experimental será compartilhado entre a linha de base baseada em regras e a abordagem apoiada por LLM. As condições de equivalência entre observabilidade, estado operacional, espaço de ações, validação e execução são detalhadas nas subseções Subseção 4.3.2 a Subseção 4.3.12.

A escolha por um ambiente simulado é compatível com o delineamento experimental da pesquisa, pois permite maior controle das variáveis observadas e repetição dos cenários sob

condições equivalentes. Conforme o material metodológico anexado, a pesquisa experimental exige a definição do objeto de estudo, das variáveis que o influenciam e das formas de controle e observação dos efeitos produzidos.

A comunicação assíncrona será implementada utilizando RabbitMQ, adotando mecanismos padronizados de tratamento de falhas para ambas as abordagens experimentais. Cada mensagem possuirá identificador único, utilizado para detectar reentregas e evitar processamento duplicado. Será estabelecido previamente um limite máximo de tentativas de processamento e um intervalo padronizado entre tentativas. Mensagens que excederem o número máximo de tentativas serão encaminhadas para uma *dead-letter queue* (DLQ), permitindo distingui-las das mensagens pendentes no fluxo normal. Esses parâmetros serão mantidos constantes nas duas abordagens para evitar que diferenças nas políticas básicas de mensageria interfiram na comparação entre os mecanismos de decisão.

O envelope lógico de cada mensagem será ampliado com campos que permitem correlacionar a mensagem à tarefa e representar sua posição lógica na trajetória, conforme o Código 1.

Código 1 – Exemplo de envelope lógico versionado de mensagem

```
1 {  
2   "schema_version": "1.0",  
3   "execution_id": "EXP_0042",  
4   "message_id": "MSG_0192",  
5   "task_id": "TASK_0187",  
6   "event_type": "STOCK_RESERVATION_REQUESTED",  
7   "event_seq": 2,  
8   "attempt_number": 1,  
9   "published_at": "2026-09-10T14:32:11.000Z",  
10  "decision_id": "DEC_0091",  
11  "target": "inventory.primary",  
12  "payload": {  
13    "order_id": "ORD_0187",  
14    "items": [  
15      {  
16        "sku": "SKU-001",  
17        "quantity": 2  
18      }  
19    ]  
20  }  
21 }
```

FONTE: Elaborada pelo autor com base no delineamento experimental desta pesquisa.

O campo “message_id” permite detectar reentregas; “task_id” correlaciona a mensagem à tarefa; “event_type” indica a transição representada; “event_seq” representa a ordem lógica local da tarefa; e “published_at” registra o tempo físico de publicação. O campo “event_seq” não deve ser denominado relógio de Lamport: trata-se apenas de uma sequência

de aplicação por tarefa. Além desses, o campo “*schema_version*” identifica a versão do contrato lógico da mensagem; “*execution_id*” associa a mensagem à execução experimental; “*attempt_number*” representa a tentativa lógica do fluxo; “*decision_id*” permite relacionar uma nova mensagem à decisão que a originou; “*target*” identifica a rota lógica selecionada; e “*payload*” contém os dados mínimos necessários ao processamento da solicitação.

Para fins experimentais, não será pressuposta uma garantia de ordenação global entre todas as mensagens do sistema. A ordem semanticamente relevante será controlada por tarefa. Cada evento pertencente a uma mesma “*task_id*” receberá um “*event_seq*” monotonicamente crescente, permitindo identificar a sequência lógica prevista para aquela trajetória. Reentregas manterão o mesmo “*message_id*” e o mesmo “*event_seq*” da mensagem original. Essa informação será registrada independentemente dos “*timestamps*” físicos e permitirá diferenciar duplicação, reentrega e eventual observação fora da sequência esperada.

Para fins deste estudo, será diferenciada a reentrega de uma mesma mensagem pelo mecanismo de mensageria de uma nova tentativa lógica selecionada pelo orquestrador. Em uma reentrega, “*message_id*”, “*task_id*”, “*event_seq*” e “*attempt_number*” permanecerão associados à mensagem original. Em uma ação RETRY, por outro lado, será produzida uma nova mensagem, com novo “*message_id*” e novo “*event_seq*”, mantendo-se a mesma “*task_id*” e incrementando-se “*attempt_number*”. Essa distinção impede que mecanismos de entrega do “*broker*” sejam interpretados como decisões de recuperação do orquestrador.

Além da identificação de reentregas por “*message_id*”, o Serviço de Estoque adotará proteção de idempotência no nível da tarefa. Isso é necessário porque uma nova tentativa lógica originada por RETRY utiliza novo “*message_id*”. Assim, o processamento deverá impedir que uma “*task_id*” cuja reserva já tenha sido efetivada produza uma segunda reserva. Essa proteção permite distinguir a idempotência de transporte, associada à mensagem, da idempotência do efeito de negócio, associada à tarefa.

Para evitar que diferenças na infraestrutura básica de tolerância a falhas interfiram na comparação entre os mecanismos decisórios, as duas abordagens utilizarão os mesmos limites operacionais e mecanismos de proteção definidos para o ambiente experimental. Os valores de timeout, limite total de tentativas, tratamento de mensagens após falhas sucessivas e isolamento de recursos serão previamente configurados e mantidos constantes entre as execuções. Esses mecanismos atuarão como restrições comuns às duas abordagens, enquanto a seleção de ações como CONTINUE, RETRY, WAIT, FALLBACK ou ABORT constituirá responsabilidade do mecanismo de orquestração correspondente.

4.3.1 Escolha do Caso

A escolha do caso justifica-se pela aderência entre o problema investigado e o contexto arquitetural dos microserviços. Microserviços são adequados para este estudo porque envolvem comunicação distribuída, implantação independente, falhas parciais e necessidade de coorde-

nação entre serviços. Fowler e Lewis (2017) destacam que microserviços são componentes independentemente substituíveis e atualizáveis, comunicando-se por mecanismos leves. Newman (2015) também aponta que microserviços favorecem limites modulares fortes, implantação independente e diversidade tecnológica.

O ambiente será composto por dois microserviços que representam uma jornada fictícia e simplificada de processamento de pedidos. O Serviço de Pedidos será responsável por receber, validar e registrar uma solicitação, enquanto o Serviço de Estoque realizará a reserva simulada dos itens correspondentes. Cada serviço manterá persistência própria em banco SQLite local, sem compartilhamento direto de dados entre os componentes. A comunicação entre os serviços será assíncrona e intermediada pelo RabbitMQ. Essa configuração permite representar dependência entre serviços, falhas parciais, reentregas, repetição de tentativas e recuperação, preservando ao mesmo tempo um escopo compatível com a implementação experimental.

Para preservar o recorte de dois microserviços, os componentes responsáveis pela construção do estado e pela coordenação decisória não constituirão um terceiro serviço de negócio. O “*State Builder*”, o “*DecisionEngine*”, o “*DecisionValidator*” e o “*DecisionExecutor*” serão implementados como módulos da camada de coordenação associada ao Serviço de Pedidos. O Serviço de Estoque permanecerá responsável pela reserva simulada e por sua própria persistência, sem acesso direto ao banco do Serviço de Pedidos.

O fluxo funcional normal será iniciado pela criação de um pedido no Serviço de Pedidos. Após validação e persistência local, será criada uma tarefa identificada por “*task_id*”. O estado correspondente será fornecido ao mecanismo decisório e, na trajetória normal, a ação CONTINUE resultará na publicação de uma solicitação de reserva para o Serviço de Estoque. Após processar a reserva e persistir seu resultado, o Serviço de Estoque publicará um evento de retorno, que será consumido pelo Serviço de Pedidos para atualizar a tarefa e concluir o pedido. Eventos terminais de sucesso não exigirão nova decisão. Em síntese, o percurso é: <POST/orders> → validação → persistência do pedido e da tarefa → “*State Builder*” → “*DecisionEngine*” → CONTINUE → “*RabbitMQ*” (*inventory.primary*) → Serviço de Estoque → persistência da reserva → STOCK_RESERVATION_SUCCEEDED → “*RabbitMQ*” (*orders.events*) → Serviço de Pedidos → COMPLETED.

A escolha pela orquestração baseada em regras como linha de base também se justifica pela literatura. Richardson (2019) organiza padrões de microserviços para lidar com problemas recorrentes de comunicação, transações distribuídas, *Saga* e consistência entre serviços, demonstrando que a coordenação de fluxos distribuídos é um problema central desse tipo de arquitetura. Assim, a comparação com um agente LLM permite investigar se uma camada decisória mais adaptativa pode apresentar benefícios em cenários de falha e variação.

Para tornar explícita a trajetória operacional, cada tarefa será representada por um estado interno. Serão utilizados os estados PENDING, DISPATCHED, PROCESSING, WAITING, RETRYING, FALLBACK_PROCESSING, COMPLETED, ABORTED e DEAD_LETTERED. Os estados COMPLETED,

ABORTED e DEAD_LETTERED serão terminais. No nível do pedido, a representação externa será simplificada em PENDING, COMPLETED e FAILED. Essa separação permite registrar decisões e recuperações intermediárias sem ampliar desnecessariamente o domínio de negócio, conforme sintetizado no Quadro 1.

Quadro 1 – Estados internos da tarefa e representação externa do pedido

Estado interno da tarefa	Terminal	Significado
PENDING	Não	Tarefa criada e ainda não despachada.
DISPATCHED	Não	Solicitação publicada para o Serviço de Estoque.
PROCESSING	Não	Reserva em processamento no Serviço de Estoque.
WAITING	Não	Novo despacho suspenso temporariamente após ação WAIT.
RETRYING	Não	Nova tentativa lógica em andamento após ação RETRY.
FALLBACK_PROCESSING	Não	Processamento pela rota <code>inventory.fallback</code> após ação FALLBACK.
COMPLETED	Sim	Reserva efetivada e pedido concluído.
ABORTED	Sim	Tarefa encerrada de forma controlada após ação ABORT.
DEAD_LETTERED	Sim	Mensagem encaminhada à <i>dead-letter queue</i> após falhas sucessivas.
Pedido (representação externa)	PENDING, COMPLETED ou FAILED	

FONTE: FONTE: Elaborado pelo autor com base no delineamento experimental desta pesquisa.

4.3.2 Fluxo de Observabilidade, Decisão e Atuação

A tomada de decisão proposta neste trabalho depende da disponibilidade de informações sobre o estado corrente do ambiente experimental. Entretanto, a utilização de um agente baseado em LLM não implica que o modelo tenha acesso direto e irrestrito aos componentes da infraestrutura, aos arquivos de “log” ou às interfaces de monitoramento. Para preservar o controle experimental e reduzir diferenças entre as abordagens comparadas, será adotada uma camada intermediária responsável por coletar, correlacionar e estruturar os sinais relevantes antes que estes sejam apresentados ao mecanismo de decisão.

Essa separação também permite distinguir os dados utilizados para observação operacional durante a execução daqueles coletados exclusivamente para análise posterior dos experimentos. Métricas como latência, estado dos serviços e quantidade de mensagens pendentes podem participar da tomada de decisão durante a execução, enquanto indicadores agregados, como latência P95 ou estatísticas consolidadas entre repetições, serão calculados posteriormente e não constituirão informação disponível ao orquestrador. Essa distinção evita que os mecanismos decisórios utilizem informações produzidas somente após a conclusão do cenário experimental.

A literatura utilizada neste trabalho reforça a necessidade de intermediar a telemetria operacional antes de sua utilização por agentes. O *KubeIntellect* emprega “*pipelines*” para coletar e processar “*logs*”, métricas e sinais de saúde do ambiente, além de manter estados intermediários e contexto persistente ao longo dos fluxos de execução (ARDEBILI; BARTOLINI, 2025). De forma semelhante, o *ServiceOdyssey* utiliza uma camada de dados intermediária entre o ambiente de microserviços e os componentes baseados em LLM, armazenando o estado corrente do sistema e o histórico de interação utilizado durante o gerenciamento autônomo (YU et al., 2025).

TABELA 7 – Camadas de dados do fluxo de observabilidade

Camada	Exemplos de dados	Utilização	Consumidor
Telemetria bruta	“Logs” dos serviços, eventos “ <i>RabbitMQ</i> ”, “ <i>timestamps</i> ”, CPU, memória, estado das filas.	Registrar o comportamento do ambiente.	Coletores e módulo de construção do estado.
Estado operacional normalizado	Serviço atual, disponibilidade, latência, tamanho da fila, tentativa atual, último resultado.	Suportar a tomada de decisão.	Orquestrador baseado em regras e agente LLM.
Registro decisório	Ação proposta, alvo, validação, ação executada, tempo de decisão.	Rastreabilidade e auditoria do fluxo.	Orquestrador e coleta experimental.
Dados analíticos	P95, “ <i>throughput</i> ”, taxa de erros, tempo de recuperação, “ <i>blast radius</i> ”.	Comparação estatística após as execuções.	Procedimentos de análise.

FONTE: Elaborada pelo autor com apoio da ferramenta Chat GPT 5.5, com base em Ardebili e Bartolini (2025), Yu et al. (2025) e no delineamento experimental desta pesquisa.

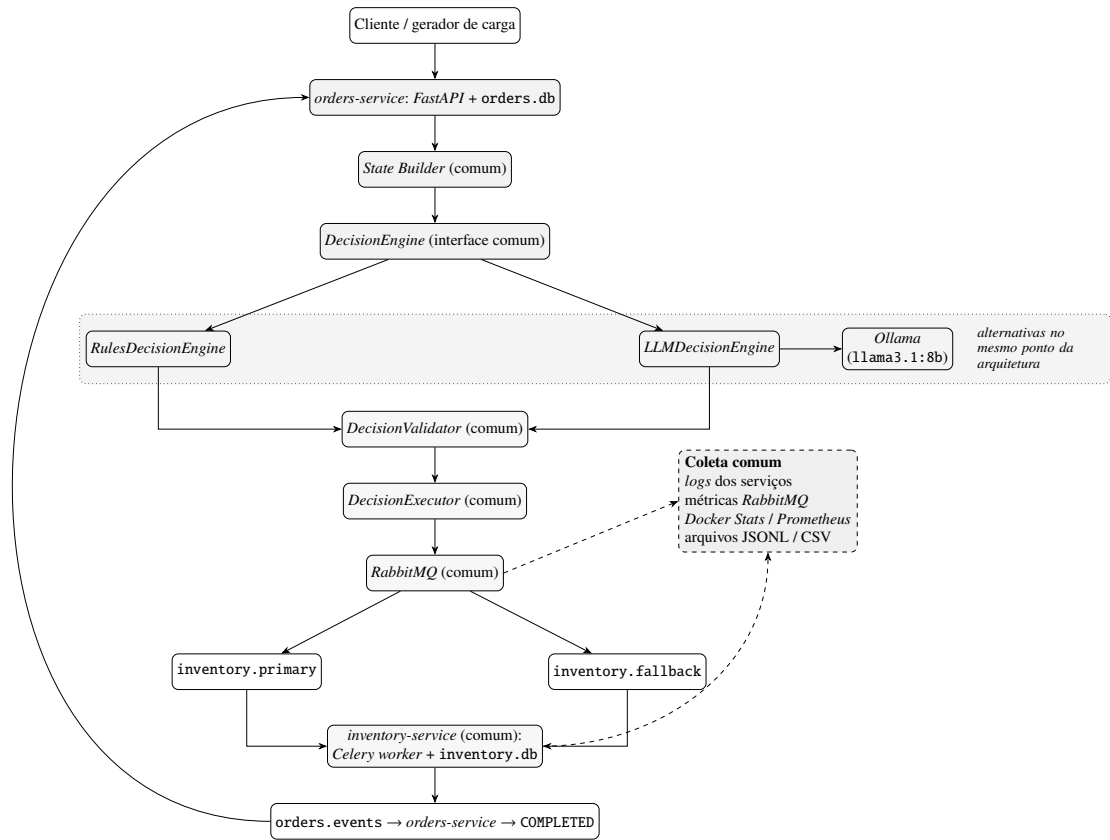
Para o presente experimento, essa organização será simplificada e implementada no próprio mecanismo de orquestração, sem introduzir uma nova plataforma de observabilidade ou um agente adicional. Os dados serão organizados em quatro níveis, apresentados na Tabela 7.

A Tabela 7 evidencia que a quantidade de dados armazenados pelo experimento será maior que a quantidade de informações fornecidas ao mecanismo decisório. Dessa forma, o fato de uma métrica ser coletada não implica necessariamente que ela seja utilizada pelo agente LLM ou pelo orquestrador baseado em regras. Essa delimitação é necessária para evitar que uma das abordagens tenha acesso a informações indisponíveis à outra e para preservar a equivalência das condições experimentais.

Essa diferença também estabelece uma separação entre histórico experimental e contexto decisório. O histórico completo de eventos, estados e decisões será preservado nos arquivos de coleta, mas somente o subconjunto previamente definido como operacionalmente relevante integrará o “SYSTEM_STATE”. Desse modo, a arquitetura não dependerá da capacidade do LLM de manter autonomamente toda a trajetória em sua própria janela de contexto.

4.3.3 Fluxo geral de observação e decisão

FIGURA 7 – Arquitetura de software e fluxo de observação, decisão e atuação



FONTE: Elaborada pelo autor com base no delineamento experimental desta pesquisa.

O processo completo de obtenção do estado e execução de uma decisão é representado na Figura 7. Os microserviços e o RabbitMQ produzem sinais operacionais durante a execução. Esses dados são coletados e convertidos em uma representação padronizada do estado, denominada neste trabalho “*SYSTEM_STATE*”. A mesma representação é fornecida ao motor de regras ou ao agente LLM, de acordo com a abordagem experimental em execução. A decisão produzida passa por validação determinística antes de ser encaminhada ao executor. Após a execução, seus efeitos voltam a ser observados pelo sistema, possibilitando novos ciclos de decisão.

A Figura 7 também explicita uma condição central da comparação experimental: os dois mecanismos decisórios compartilham a mesma camada de observação, a mesma representação do estado, o mesmo conjunto de ações permitidas, o mesmo mecanismo de validação e o mesmo executor. A diferença entre as abordagens será concentrada no componente responsável por selecionar uma ação. Na linha de base tradicional, a escolha será determinada por regras e limiares previamente especificados; na abordagem experimental, a escolha será produzida pelo agente LLM a partir do estado operacional estruturado.

Essa organização reduz o risco de atribuir ao uso do LLM diferenças decorrentes, na realidade, de observabilidade ou infraestrutura distintas.

4.3.4 Construção do estado operacional

O componente responsável pela conversão da telemetria em uma representação utilizada pelo decisor será denominado “*State Builder*”. Sua função será consultar as informações necessárias associadas à tarefa em execução, correlacioná-las por identificadores e construir uma estrutura padronizada antes de cada decisão. Esse componente não realizará escolha de ação, funcionando apenas como uma interface entre observabilidade e decisão.

Para evitar que informações excessivas ou não comparáveis sejam fornecidas ao agente, o estado será composto predominantemente por variáveis previamente definidas. Essa decisão também reduz a necessidade de fornecer grandes volumes de “logs” textuais ao LLM. Cornacchia et al. (2025) identificam justamente o uso ineficiente de contexto como uma das limitações observadas em agentes aplicados à análise de falhas em microserviços. Pei et al. (2025) também destacam que observações complexas e grandes espaços de ação dificultam a seleção adequada de ações pelos agentes.

O “*State Builder*” também funcionará como mecanismo explícito de controle de contexto. O histórico integral da tarefa não será concatenado ao “*prompt*” a cada decisão. O campo “*recent_events*” conterá uma janela limitada e ordenada de eventos, cujo tamanho máximo será definido antes da coleta definitiva e mantido constante nas duas abordagens. Eventos anteriores permanecerão preservados nos registros experimentais, mas não serão automaticamente reenviados ao decisor. Essa distinção reduz crescimento descontrolado do contexto e torna explícita a informação efetivamente disponível em cada ponto de decisão.

O valor de K deverá ser definido antes da coleta definitiva. O campo “*timestamp*” já existente deve ser mantido e interpretado como o horário de construção do “*snapshot*”.

O estado distingue o serviço físico da rota lógica: “*current_service*” identifica o microserviço envolvido (“*inventory-service*”), enquanto “*current_target*” identifica a rota lógica em uso (“*inventory.primary*” ou “*inventory.fallback*”). Essa separação é necessária porque “*inventory.primary*” e “*inventory.fallback*” são rotas do mesmo microserviço, e não serviços distintos. O campo “*alternative_targets*” lista os alvos lógicos admissíveis pela configuração do fluxo.

A composição exata poderá ser refinada durante a implementação, desde que os campos utilizados sejam definidos antes da execução dos experimentos e mantidos equivalentes entre as duas abordagens. Variáveis adicionais, como utilização de CPU ou memória, somente deverão integrar o estado decisório quando fizerem parte explicitamente da lógica de determinado cenário, como na avaliação de sobrecarga. Caso contrário, permanecerão disponíveis apenas para análise posterior.

O Quadro 8 apresenta um exemplo da representação produzida pelo State Builder diante de uma tarefa cujo processamento apresentou sucessivos “*timeouts*”.

O código acima 2 demonstra que o LLM não receberá a indicação explícita da falha

Código 2 – Exemplo de representação normalizada do estado operacional estruturado (SYSTEM_STATE)

```
1 {
2   "execution_id": "EXP_0042",
3   "task_id": "TASK_0187",
4   "state_id": "STATE_0091",
5   "timestamp": "2026-09-10T14:32:11.512Z",
6   "current_event_seq": 4,
7
8   "task": {
9     "phase": "RETRYING",
10    "current_service": "inventory-service",
11    "current_target": "inventory.primary",
12    "attempt_number": 2,
13    "max_attempts": 3,
14    "wait_count": 0,
15    "max_waits": 2,
16    "elapsed_ms": 4200
17  },
18
19  "service": {
20    "status": "degraded",
21    "latency_ms": 2054,
22    "last_result": "timeout"
23  },
24
25  "messaging": {
26    "queue_size": 4,
27    "redelivered": false
28  },
29
30  "alternatives": {
31    "fallback_available": true,
32    "fallback_used": false,
33    "alternative_targets": [
34      "inventory.fallback"
35    ]
36  },
37
38  "recent_events": [
39    {
40      "event_type": "STOCK_RESERVATION_REQUESTED",
41      "attempt_number": 2,
42      "event_seq": 3
43    },
44    {
45      "event_type": "INVENTORY_TIMEOUT",
46      "attempt_number": 2,
47      "event_seq": 4
48    }
49  ]
50 }
```

FONTE: Elaborada pelo autor com apoio da ferramenta Chat GPT 5.5, com base no delineamento experimental desta pesquisa.

TABELA 8 – Campos do estado operacional construído pelo *State Builder*

Campo	Tipo	Origem	Função na decisão
execution_id	texto	executor experimental	Identificar a execução.
task_id	texto	produtor/orquestrador	Correlacionar eventos da mesma tarefa.
state_id	texto	<i>State Builder</i>	Identificar de forma única o “ <i>snapshot</i> ” fornecido ao decisor.
timestamp	data/hora	orquestrador	Horário de construção do “ <i>snapshot</i> ”.
current_event_seq	inteiro	histórico da tarefa	Identificar a posição lógica atual da tarefa.
phase	categorico	orquestrador	Identificar a fase corrente da tarefa.
current_service	texto	orquestrador	Identificar o serviço físico atual.
current_target	texto	orquestrador	Identificar a rota lógica atualmente utilizada.
attempt_number	inteiro	orquestrador	Informar o número de tentativas já realizadas.
max_attempts	inteiro	configuração experimental	Informar ao decisor o limite total de tentativas.
elapsed_ms	número	“ <i>timestamps</i> ”	Informar tempo já transcorrido.
service_status	categorico	serviço/orquestrador	Indicar estado observado do serviço.
latency_ms	número	métricas/“ <i>logs</i> ”	Representar degradação de resposta.
last_result	categorico	“ <i>logs</i> ” da tarefa	Informar resultado da última tentativa.
queue_size	inteiro	<i>RabbitMQ</i>	Representar pressão/acúmulo da fila.
redelivered	booleano	mensageria	Identificar reentrega.
wait_count	inteiro	orquestrador	Informar quantas ações WAIT já foram realizadas.
max_waits	inteiro	configuração experimental	Limitar espera indefinida.
fallback_available	booleano	configuração do fluxo	Informar existência de alternativa válida.
fallback_used	booleano	histórico/orquestrador	Impedir repetição indefinida do <i>fallback</i> .
alternative_targets	lista	configuração do fluxo	Informar alvos lógicos admissíveis.
recent_events	lista limitada e ordenada	histórico da tarefa	Fornecer as <i>K</i> ocorrências mais recentes definidas pela política experimental.

FONTE: Elaborada pelo autor com apoio da ferramenta Chat GPT 5.5, com base no delineamento experimental desta pesquisa.

artificial introduzida pelo pesquisador nem informações sobre o comportamento futuro do cenário. O agente observa apenas os efeitos operacionalmente disponíveis, como “*timeout*”, degradação, quantidade de tentativas e estado da fila. O campo “*state_id*” identifica de forma única esse “*snapshot*”, e o campo “*recent_events*” não representa o histórico completo da tarefa, mas apenas a janela de eventos atualmente selecionada pelo “*State Builder*”.

Essa distinção é relevante para evitar vazamento de informação experimental. O componente responsável pela injeção de falha poderá conhecer, por exemplo, que a rota “*inventory.primary*” permanecerá degradada por determinado período, mas o mecanismo decisório deverá observar apenas aquilo que seria obtido por meio dos sinais disponibilizados pelo próprio ambiente.

4.3.5 Pontos de decisão

A existência de telemetria disponível não implica que uma nova inferência LLM seja executada continuamente. Para controlar o número de chamadas ao modelo, reduzir custo e aumentar a reprodutibilidade, serão estabelecidos previamente eventos capazes de gerar um novo ponto de decisão.

Não se apresenta FALLBACK como ação possível quando o “*inventory-service*” inteiro estiver indisponível, pois a rota de *fallback* permanece dentro desse mesmo serviço.

TABELA 9 – Separação entre estado experimental e estado observável

Informação	Disponível ao pesquisador	Disponível ao decisor
Tipo de falha injetada	Sim	Não diretamente
Duração programada da falha	Sim	Não
Serviço observado como indisponível	Sim	Sim
Ocorrência de “ <i>timeout</i> ”	Sim	Sim
Número de tentativas realizadas	Sim	Sim
Tamanho atual da fila	Sim	Sim
Tempo restante até recuperação induzida	Sim	Não
Resultado futuro esperado	Sim, após análise	Não

FONTE: Elaborada pelo autor com apoio da ferramenta Chat GPT 5.5, com base no delineamento experimental desta pesquisa.

TABELA 10 – Eventos previstos para acionamento do mecanismo decisório

Evento observado	Nova decisão	Ações admissíveis
Início da tarefa	Sim	CONTINUE
Conclusão normal terminal	Não	nenhuma
“ <i>Timeout</i> ”	Sim	RETRY, WAIT, FALLBACK, ABORT
Falha transitória	Sim	RETRY, WAIT, FALLBACK, ABORT
Serviço totalmente indisponível	Sim	WAIT, ABORT
Dados inválidos ou inconsistentes	Sim	ABORT
Conclusão de nova tentativa	Conforme resultado	CONTINUE, RETRY, FALLBACK ou ABORT, conforme estado
Atualização isolada de métrica sem alteração relevante	Não	nenhuma

FONTE: Elaborada pelo autor com apoio da ferramenta Chat GPT 5.5, com base no delineamento experimental desta pesquisa.

Para reduzir ambiguidades de implementação, as ações admitidas possuirão semântica previamente definida. CONTINUE executará a próxima transição normal do fluxo. RETRY produzirá uma nova tentativa lógica no mesmo *target*, mantendo “*task_id*” e incrementando “*attempt_number*”. WAIT suspenderá temporariamente novo despacho, aguardará o intervalo configurado e provocará nova construção do estado, sem incrementar “*attempt_number*”. FALLBACK substituirá a rota “*inventory.primary*” por “*inventory.fallback*”, quando essa alternativa estiver disponível e ainda não tiver sido utilizada. ABORT encerrará a tarefa de maneira controlada e produzirá estado terminal de falha.

Dessa forma, as decisões serão orientadas por eventos relevantes para o fluxo, e não pela leitura contínua de todas as métricas disponíveis. Cada novo ponto de decisão produzirá um *decision_id*, que permitirá correlacionar o estado apresentado ao decisor, a decisão produzida e seus efeitos posteriores.

4.3.6 Comparação entre motores de decisão

Após a construção do `SYSTEM_STATE`, a representação será encaminhada ao mecanismo correspondente à abordagem executada. Na linha de base, um motor determinístico verificará os campos do estado segundo políticas previamente configuradas. Na abordagem baseada em LLM, o mesmo estado será incorporado a um “*prompt*” padronizado.

Ambos os motores de decisão implementarão uma mesma interface, garantindo que a substituição entre abordagens não altere o restante do fluxo de orquestração, conforme ilustrado no Código 3.

Código 3 – Interface comum aos motores de decisão

```
1 class DecisionEngine:
2
3     def decide(self, state):
4         raise NotImplementedError
```

FONTE: Elaborada pelo autor com apoio da ferramenta Chat GPT 5.5, com base no delineamento experimental desta pesquisa.

Conceitualmente, a interface poderá ser representada pelo código 3.

A política do “*RulesDecisionEngine*” será definida antes da coleta definitiva e permanecerá inalterada durante as repetições. As regras serão avaliadas em ordem de prioridade previamente estabelecida, utilizando exclusivamente os campos do “`SYSTEM_STATE`” e os mesmos limites operacionais disponibilizados à abordagem baseada em LLM. O parâmetro “`max_attempts`” representará o número total de tentativas permitidas, incluindo a tentativa inicial; assim, “`max_attempts = 3`” corresponde a uma tentativa inicial e, no máximo, duas novas tentativas. O Código 4 apresenta a lógica fechada dessa política.

O código explicita que a abordagem tradicional também dispõe de mecanismos de recuperação e que a diferença investigada não será a simples existência de “*retry*” ou “*fall-back*”, mas a forma pela qual uma ação é selecionada diante do estado observado. Os valores numéricos concretos dos limiares — “`timeout`”, “`max_attempts`”, “`wait_delay`”, “`max_waits`”, “`queue_high_watermark`” e “`task_deadline`” — não precisam ser definitivos antes do piloto: a lógica permanece fechada, enquanto esses valores poderão ser ajustados tecnicamente durante a fase de piloto e serão congelados antes da coleta definitiva.

4.3.7 Construção do “*prompt*” do agente LLM

Na abordagem experimental, o `SYSTEM_STATE` será incorporado a um modelo de “*prompt*” fixo. A padronização é necessária porque a sensibilidade à formulação do “*prompt*” já é reconhecida pelo próprio experimento como uma limitação da abordagem baseada em LLM. O texto permanente do “*prompt*”, o conjunto de ações possíveis e as restrições operacionais permanecerão constantes entre as execuções; somente o estado observado será atualizado.

Código 4 – Política determinística do *RulesDecisionEngine*

```
1 def decide(state):
2
3     if state["task"]["elapsed_ms"] >= TASK_DEADLINE_MS:
4         return {"action": "ABORT", "target": None,
5                 "reason_code": "TASK_DEADLINE_EXCEEDED"}
6
7     if state["service"]["last_result"] == "invalid_data":
8         return {"action": "ABORT", "target": None,
9                 "reason_code": "INVALID_DATA"}
10
11    if state["service"]["last_result"] == "fallback_failed":
12        return {"action": "ABORT", "target": None,
13                "reason_code": "FALLBACK_FAILED"}
14
15    if state["service"]["status"] == "unavailable":
16        if state["task"]["wait_count"] < state["task"]["max_waits"]:
17            return {"action": "WAIT", "target": None,
18                    "reason_code": "SERVICE_UNAVAILABLE"}
19        return {"action": "ABORT", "target": None,
20                "reason_code": "SERVICE_UNAVAILABLE_LIMIT"}
21
22    if state["messaging"]["queue_size"] >= QUEUE_HIGH_WATERMARK:
23        if state["task"]["wait_count"] < state["task"]["max_waits"]:
24            return {"action": "WAIT", "target": None,
25                    "reason_code": "QUEUE_PRESSURE"}
26        return {"action": "ABORT", "target": None,
27                "reason_code": "QUEUE_PRESSURE_LIMIT"}
28
29    if state["service"]["last_result"] in {"timeout",
30        "transient_error"}:
31        if state["task"]["attempt_number"] <
32            state["task"]["max_attempts"]:
33            return {"action": "RETRY",
34                    "target": state["task"]["current_target"],
35                    "reason_code": "TRANSIENT_RETRY"}
36        if (state["alternatives"]["fallback_available"]
37            and not state["alternatives"]["fallback_used"]):
38            return {"action": "FALLBACK", "target":
39                    "inventory.fallback",
40                    "reason_code": "PRIMARY_EXHAUSTED"}
41        return {"action": "ABORT", "target": None,
42                "reason_code": "ATTEMPTS_EXHAUSTED"}
43
44    if state["task"]["phase"] in {"PENDING", "READY", "RECOVERED"}:
45        return {"action": "CONTINUE", "target": "inventory.primary",
46                "reason_code": "NORMAL_FLOW"}
47
48    return {"action": "ABORT", "target": None,
49            "reason_code": "UNMAPPED_STATE"}
```

FONTE: Elaborada pelo autor com apoio da ferramenta Chat GPT 5.5, com base no delineamento experimental desta pesquisa.

O agente será tratado como *stateless* entre pontos de decisão para fins experimentais: a continuidade da tarefa não dependerá de memória conversacional interna acumulada pelo modelo, mas das informações reconstruídas externamente pelo “*State Builder*”. Essa escolha reduz dependência de histórico implícito do “*runtime*” e preserva a equivalência entre execuções. Esse tratamento pressupõe que a implementação realize chamadas independentes ao modelo, reconstruindo o “*prompt*” a cada decisão; caso o “*runtime*” escolhido mantenha sessão conversacional automaticamente, essa função deverá ser desativada ou documentada.

Quadro 2 – Modelo fixo do “*prompt*” do agente LLM

PAPEL

Você atua como mecanismo de decisão de um orquestrador em um ambiente experimental de microserviços.

OBJETIVO

Selecionar a próxima ação válida utilizando exclusivamente o estado operacional fornecido.

AÇÕES PERMITIDAS

- CONTINUE
- RETRY
- WAIT
- FALLBACK
- ABORT

RESTRIÇÕES

- Não inventar serviços, targets ou recursos inexistentes.
- Não ultrapassar o limite máximo de tentativas.
- Utilizar somente alternativas fornecidas no estado.
- Selecionar apenas uma ação pertencente ao conjunto permitido.
- Responder exclusivamente no formato solicitado.

ESTADO OPERACIONAL

{{SYSTEM_STATE}}

FORMATO OBRIGATÓRIO DA RESPOSTA

```
{
  "action": "<AÇÃO>",
  "target": "<TARGET ou null>",
  "reason_code": "<CÓDIGO>"
}
```

FONTE: Fonte: Elaborada pelo autor com apoio da ferramenta Chat GPT 5.5, com base no delineamento experimental desta pesquisa.

O campo “target” é descrito como “<TARGET ou null>”, e não como serviço, porque

o *fallback* é uma rota lógica do mesmo Serviço de Estoque, e não um terceiro serviço.

A fase de piloto será iniciada utilizando o “*Ollama*” como “*runtime*” local e o modelo “*llama3.1:8b*”. Caso essa configuração se mostre tecnicamente inviável no “*hardware*” disponível, eventual substituição ocorrerá exclusivamente durante a fase de piloto. Antes da primeira execução integrante da amostra, o modelo definitivo, seu identificador ou *digest*, a quantização, a versão do “*runtime*” e os parâmetros de inferência serão congelados e registrados nos metadados de execução. A configuração inicial do piloto adotará “*runtime = ollama*”, “*model = llama3.1:8b*”, “*temperature = 0.0*”, “*top_p = 1.0*”, “*max_tokens = 64*”, “*seed = 42*” (somente se efetivamente suportada), “*request_timeout_seconds = 30*”, “*session_memory = false*” e “*stream = false*”; esses valores serão tratados como configuração de piloto até serem confirmados. Na coleta definitiva, modelo, quantização, “*runtime*”, versão, parâmetros e “*prompt*” permanecerão constantes.

4.3.8 Formato da decisão

O agente não produzirá comandos arbitrários nem executará diretamente operações sobre os microserviços. A saída será convertida em um objeto de decisão estruturado, contendo somente ações reconhecidas pela arquitetura experimental.

Código 5 – Exemplo de decisão estruturada produzida pelo agente

```
1 {
2   "action": "FALLBACK",
3   "target": "inventory.fallback",
4   "reason_code": "PRIMARY_EXHAUSTED"
5 }
```

FONTE: Elaborada pelo autor com apoio da ferramenta Chat GPT 5.5, com base no delineamento experimental desta pesquisa.

O campo “*action*” deverá pertencer obrigatoriamente ao conjunto de ações permitido. O campo “*target*” será obrigatório quando a ação determinar uma rota de execução; em ações sem destino operacional, como *WAIT* e *ABORT*, seu valor será “*null*”. O campo “*reason_code*” possibilitará registrar de forma categórica o motivo associado à decisão, evitando a necessidade de analisar justificativas extensas em linguagem natural.

TABELA 11 – Contrato previsto para a resposta do mecanismo baseado em LLM

Campo	Obrigatoriedade	Conteúdo esperado
action	Obrigatório	CONTINUE, RETRY, WAIT, FALLBACK ou ABORT
target	Condicional	target lógico permitido ou null
reason_code	Obrigatório	código categórico associado à decisão

FONTE: Elaborada pelo autor com apoio da ferramenta Chat GPT 5.5, com base no delineamento experimental desta pesquisa.

Uma proposta inicial para o contrato de saída é apresentada na Tabela 11.

4.3.9 Validação e execução das decisões

A geração de uma resposta pelo mecanismo decisório não implica sua execução automática. Antes de atuar sobre o ambiente, a decisão será encaminhada a um componente determinístico de validação, comum às abordagens *Rules* e LLM. Essa etapa verificará o formato da resposta, a existência da ação solicitada, a validade do *target* de destino e o atendimento aos limites operacionais previamente definidos, como o número total de tentativas.

Essa restrição é coerente com os riscos discutidos na fundamentação do trabalho. Cornacchia et al. (2025) identificam, entre os modos de falha de agentes aplicados a microserviços, erros de parametrização, uso inadequado de ferramentas e trajetórias de raciocínio incorretas. O *KubeIntellect* também separa interpretação, validação, políticas e execução como responsabilidades distintas de sua arquitetura.

A lógica do validador poderá ser implementada de acordo com o Código 6. O mesmo validador será aplicado às abordagens *Rules* e LLM, não sendo um componente exclusivo do agente LLM.

Caso a resposta não satisfaça as restrições estabelecidas, a proposta será registrada como decisão inválida e a tarefa será encerrada por meio da ação ABORT, com “*reason_code*” igual a “INVALID_DECISION”. Não será realizada nova chamada ao LLM para autocorreção da resposta, não será utilizado o “*RulesDecisionEngine*” como *fallback* do mecanismo LLM e o executor não converterá uma ação inválida em outra ação aproximada. Essa política preserva a observação dos erros do mecanismo decisório como parte dos resultados experimentais.

Uma decisão inválida produzida pelo mecanismo LLM não será, por si só, critério de invalidação da execução experimental. Nesse caso, a decisão será rejeitada, a ação ABORT será executada e a ocorrência permanecerá na amostra como comportamento observado da abordagem, com “*validation.valid = false*”, “*executed_decision = ABORT*” e “*run_status = VALID*”. Execuções serão classificadas como inválidas apenas quando ocorrerem problemas da bancada experimental que impeçam a aplicação íntegra do protocolo, como falha de inicialização de componentes obrigatórios, ausência de “*readiness*”, restauração incorreta do estado inicial, corrupção de arquivos essenciais ou inconsistência da configuração; nesses casos, “*run_status = INVALID*”, a execução não integra a amostra, o motivo é registrado e a mesma configuração é repetida.

Também será distinguida a indisponibilidade do “*runtime*” antes do início da execução de um *timeout* ocorrido durante uma chamada válida ao modelo. Se o “*runtime*” LLM não estiver disponível na verificação de “*readiness*”, a execução não será iniciada como repetição válida. Se uma chamada ao LLM exceder o limite temporal durante uma execução já iniciada sob condições válidas, a ocorrência será registrada como “LLM_DECISION_TIMEOUT” e resultará em ABORT,

Código 6 – Validação determinística das decisões

```
1 ALLOWED_ACTIONS = {
2     "CONTINUE", "RETRY", "WAIT", "FALLBACK", "ABORT"
3 }
4
5 ALLOWED_TARGETS = {
6     None, "inventory.primary", "inventory.fallback"
7 }
8
9 def validate_decision(decision, state):
10
11     if decision["action"] not in ALLOWED_ACTIONS:
12         return False, "INVALID_ACTION"
13
14     if decision.get("target") not in ALLOWED_TARGETS:
15         return False, "UNKNOWN_TARGET"
16
17     if not decision.get("reason_code"):
18         return False, "MISSING_REASON_CODE"
19
20     if decision["action"] == "RETRY":
21         if state["task"]["attempt_number"] >=
22             state["task"]["max_attempts"]:
23             return False, "RETRY_LIMIT_EXCEEDED"
24         if decision["target"] != state["task"]["current_target"]:
25             return False, "INVALID_RETRY_TARGET"
26
27     if decision["action"] == "FALLBACK":
28         if not state["alternatives"]["fallback_available"]:
29             return False, "FALLBACK_NOT_AVAILABLE"
30         if state["alternatives"]["fallback_used"]:
31             return False, "FALLBACK_ALREADY_USED"
32         if decision["target"] != "inventory.fallback":
33             return False, "INVALID_FALLBACK_TARGET"
34
35     if decision["action"] in {"WAIT", "ABORT"}:
36         if decision.get("target") is not None:
37             return False, "TARGET_NOT_ALLOWED"
38
39     if state["task"]["phase"] in {"COMPLETED", "ABORTED",
40         "DEAD_LETTERED"}:
41         return False, "TERMINAL_TASK"
42
43     return True, None
```

FONTE: Elaborada pelo autor com apoio da ferramenta Chat GPT 5.5, com base no delineamento experimental desta pesquisa.

permanecendo como comportamento operacional da abordagem.

A aplicação dessa etapa também estabelece uma separação explícita entre as responsabilidades do sistema: o mecanismo decisório propõe uma decisão, o validador verifica sua admissibilidade e o executor realiza a ação. Os microserviços permanecem responsáveis por sua própria lógica de processamento, evitando a concentração das regras de domínio no agente de orquestração.

4.3.10 Registro e rastreabilidade das decisões

Além dos “logs” operacionais já previstos na seção 4.5.1, cada ponto de decisão deverá registrar o estado conhecido pelo mecanismo decisório, a ação proposta, o resultado da validação e a ação efetivamente executada. Esse procedimento permite reconstruir posteriormente não apenas o resultado de uma tarefa, mas também as condições sob as quais determinada decisão foi tomada.

O contrato do arquivo “decisions.jsonl” será comum às duas abordagens e deverá seguir os seguintes princípios: “decision_time_ms” existe para “RULES” e para LLM; “llm_inference_ms” existe somente para LLM; a quantidade de decisões e de chamadas ao modelo é derivada dos próprios registros; “tokens” são registrados somente quando o “runtime” permitir obtê-los de forma confiável; e o tempo de carregamento inicial do modelo não deve ser misturado ao tempo normal de decisão.

Para a abordagem baseada em regras, será mantida estrutura equivalente, conforme o Código 8, substituindo-se “decision_engine” por “RULES” e registrando o tempo correspondente à avaliação das regras. Os campos “llm_inference_ms” e “token_usage” são aplicáveis exclusivamente à abordagem baseada em LLM, permanecendo nulos para “RULES”, enquanto “decision_time_ms” permitirá comparar o custo total de decisão entre as duas abordagens.

O Código 9 ilustra o registro de uma proposta inválida produzida pelo LLM e a consequente execução de ABORT. O valor “service_c” aparece apenas para ilustrar uma proposta que referencia um destino inexistente; nos exemplos de decisões válidas são utilizados “inventory.primary” ou “inventory.fallback”.

O campo “decision_time_ms” é definido como o tempo entre a disponibilização do “SYSTEM_STATE” ao motor decisório e a obtenção de uma ação válida e executável após a validação comum às duas abordagens. Essa definição permite incorporar, no caso LLM, a inferência, o “parsing” e a eventual rejeição imediata da resposta pelo validador. A construção do “SYSTEM_STATE” permanece fora desse tempo, por ser uma etapa compartilhada pelas duas abordagens.

A partir dos registros de “decisions.jsonl”, poderão ser calculados os seguintes indicadores derivados: “decision_count” (quantidade total de decisões na execução), “llm_call_count” (quantidade de registros cujo “decision_engine” é LLM), “total_llm_inference_

Código 7 – Exemplo de registro de decisão (*decision log*) do agente LLM

```
1 {
2   "execution_id": "EXP_0042",
3   "task_id": "TASK_0187",
4   "state_id": "STATE_0091",
5   "decision_id": "DEC_0091",
6   "decision_engine": "LLM",
7
8   "proposed_decision": {
9     "action": "RETRY",
10    "target": "inventory.primary",
11    "reason_code": "TRANSIENT_RETRY"
12  },
13
14  "validation": {
15    "valid": true,
16    "error": null
17  },
18
19  "executed_decision": {
20    "action": "RETRY",
21    "target": "inventory.primary"
22  },
23
24  "decision_time_ms": 684,
25  "llm_inference_ms": 642,
26
27  "token_usage": {
28    "input_tokens": 428,
29    "output_tokens": 21,
30    "total_tokens": 449
31  },
32
33  "timestamp": "2026-09-10T14:32:12.196Z"
34 }
```

FONTE: Elaborada pelo autor com apoio da ferramenta Chat GPT 5.5, com base no delineamento experimental desta pesquisa.

Código 8 – Exemplo de registro de decisão (*decision log*) equivalente para a abordagem baseada em regras

```
1 {
2   "execution_id": "EXP_0042",
3   "task_id": "TASK_0187",
4   "state_id": "STATE_0091",
5   "decision_id": "DEC_0091",
6   "decision_engine": "RULES",
7
8   "proposed_decision": {
9     "action": "RETRY",
10    "target": "inventory.primary",
11    "reason_code": "TRANSIENT_RETRY"
12  },
13
14  "validation": {
15    "valid": true,
16    "error": null
17  },
18
19  "executed_decision": {
20    "action": "RETRY",
21    "target": "inventory.primary"
22  },
23
24  "decision_time_ms": 1.7,
25  "llm_inference_ms": null,
26  "token_usage": null,
27
28  "timestamp": "2026-09-10T14:32:12.196Z"
29 }
```

FONTE: Elaborada pelo autor com apoio da ferramenta Chat GPT 5.5, com base no delineamento experimental desta pesquisa.

(soma de “llm_inference_ms” na execução), “total_input_tokens”, “total_output_tokens” e “total_tokens”. Não é obrigatório gravar esses agregados em cada linha do arquivo, pois podem ser calculados posteriormente a partir de “decisions.jsonl”.

A inclusão de “state_id” permite associar cada decisão ao “*snapshot*” exato que foi apresentado ao mecanismo decisório. Essa correlação é especialmente importante quando a tarefa possui sucessivos estados semelhantes ou quando o ambiente se modifica durante o tempo necessário para produzir a decisão. Opcionalmente, poderá ser derivado o indicador “state_age_ms = decision_timestamp - state_timestamp”; esse indicador não precisa se tornar uma métrica principal.

A reconstrução da trajetória utilizará duas referências temporais distintas. O “timestamp” registrará o instante observado no ambiente experimental, enquanto “event_seq” registrará a

Código 9 – Exemplo de registro de decisão inválida do agente LLM

```
1 {
2   "execution_id": "EXP_0042",
3   "task_id": "TASK_0187",
4   "state_id": "STATE_0091",
5   "decision_id": "DEC_0091",
6   "decision_engine": "LLM",
7
8   "proposed_decision": {
9     "action": "RETRY",
10    "target": "service_c",
11    "reason_code": "RETRY"
12  },
13
14  "validation": {
15    "valid": false,
16    "error": "UNKNOWN_TARGET"
17  },
18
19  "executed_decision": {
20    "action": "ABORT",
21    "target": null,
22    "reason_code": "INVALID_DECISION"
23  },
24
25  "decision_time_ms": 684,
26  "llm_inference_ms": 642,
27
28  "timestamp": "2026-09-10T14:32:12.196Z"
29 }
```

FONTE: Elaborada pelo autor com apoio da ferramenta Chat GPT 5.5, com base no delineamento experimental desta pesquisa.

posição lógica local do evento dentro da respectiva tarefa. Essa separação evita interpretar a simples ordenação de “*timestamps*” como prova de causalidade distribuída e permite identificar reentregas ou ocorrências observadas fora da sequência de negócio esperada.

Os arquivos produzidos durante cada execução poderão ser organizados conforme o Quadro 3, mantendo-se os artefatos do piloto e da coleta definitiva em subárvores distintas (“<pilot/>” e “<experiment/>”).

Os identificadores “*execution_id*”, “*task_id*” e “*decision_id*” permitirão correlacionar os diferentes arquivos e reconstruir a trajetória de cada tarefa. Essa estrutura complementa o procedimento já previsto na seção de instrumentos de coleta, que estabelece a utilização de identificadores únicos para relacionar “*logs*”, métricas e decisões pertencentes a uma mesma execução.

Quadro 3 – Estrutura de diretórios dos dados coletados por execução

```
data/
|-- pilot/
|   '-- PILOT_0001/
'-- experiment/
    '-- EXP_0042/
        |-- execution_metadata.json
        |-- initial_state.json
        |-- states.jsonl
        |-- task_events.jsonl
        |-- decisions.jsonl
        |-- fault_events.jsonl
        |-- microservices_logs.jsonl
        |-- queue_metrics.csv
        '-- container_stats.csv
```

FONTE: FONTE: Elaborada pelo autor com apoio da ferramenta Chat GPT 5.5, com base no delineamento experimental desta pesquisa.

4.3.11 Realimentação do estado após ação

Após uma decisão validada ser executada, o processamento não será considerado necessariamente encerrado. Os efeitos produzidos pela ação serão novamente observados pelos componentes de coleta e incorporados ao estado operacional quando houver novo evento relevante. Assim, uma tentativa adicional pode resultar em sucesso, nova falha, aumento de fila ou alteração de disponibilidade, gerando eventualmente outro ponto de decisão.

Código 10 – Laço de realimentação do estado após a execução de uma decisão

```
1 while not task.finished:
2
3     event = wait_for_relevant_event(task.id)
4
5     if event.requires_decision:
6
7         state = build_state(task.id)
8
9         decision = decision_engine.decide(state)
10
11        valid, error = validate_decision(
12            decision,
13            state
14        )
15
16        action = resolve_action(
17            decision,
18            valid,
19            error
20        )
21
22        log_decision(
23            state,
24            decision,
25            valid,
26            action
27        )
28
29        execute(action)
```

FONTE: Elaborada pelo autor com apoio da ferramenta Chat GPT 5.5, com base no delineamento experimental desta pesquisa.

O comportamento geral pode ser representado conceitualmente pelo Código 10.

O “*DecisionExecutor*” receberá apenas decisões previamente submetidas ao mecanismo comum de validação. Sua função será traduzir uma ação admissível em uma operação prevista pelo ambiente, sem reinterpretar o conteúdo da decisão ou selecionar uma política alternativa. Quando a validação retornar resultado válido, será executada a ação proposta; quando retornar resultado inválido, a ação efetiva será ABORT com “reason_code” igual a “INVALID_DECISION”. Formalmente, “resolve_action(decision, valid, error)” devolve “decision” quando “valid” é verdadeiro e, caso contrário, “{action: ABORT, target: null, reason_code: INVALID_DECISION}”, sem outras ramificações.

Após uma decisão validada ser executada, o processamento não será considerado necessariamente encerrado. Os efeitos produzidos pela ação serão novamente observados pelos componentes de coleta e incorporados ao estado operacional quando houver novo evento relevante.

Assim, uma tentativa adicional pode resultar em sucesso, nova falha, aumento de fila ou alteração de disponibilidade, gerando eventualmente outro ponto de decisão.

A realimentação também estabelece que a continuidade lógica da tarefa será mantida externamente ao LLM. Após cada evento relevante, o “*State Builder*” produzirá um novo “*snapshot*” a partir das fontes observáveis e do histórico persistido pela aplicação. O modelo não deverá ser utilizado como repositório autoritativo do estado da execução. Dessa forma, informações como tentativa atual, último resultado e eventos recentes serão reconstruídas pelo sistema e novamente fornecidas ao mecanismo decisório quando necessário.

4.3.12 Delimitação

A estrutura proposta busca isolar o mecanismo de tomada de decisão como principal diferença entre as duas abordagens experimentais. A coleta da observabilidade, a construção do estado, o conjunto de ações disponíveis, as restrições operacionais, a validação e a execução permanecerão equivalentes entre a orquestração baseada em regras e a orquestração apoiada por LLM. Dessa forma, diferenças observadas nas métricas e no comportamento de recuperação poderão ser analisadas prioritariamente em função da estratégia utilizada para selecionar ações diante das mesmas condições operacionais, reduzindo a interferência de diferenças estruturais entre os ambientes comparados.

A arquitetura não pressupõe ordenação total global de eventos, não utiliza relógios lógicos para inferência de causalidade e não replica o mecanismo decisório. Essas propriedades são deliberadamente excluídas porque não constituem variáveis necessárias à comparação entre *Rules* e LLM no ambiente reduzido proposto.

A execução assíncrona por “*Celery*” e “*RabbitMQ*” também será comum às duas abordagens e não constituirá mecanismo decisório. Da mesma forma, a política de idempotência, o contrato de mensagens e as regras de validação permanecerão idênticos. Dessa forma, a diferença experimental será concentrada na seleção da ação pelo “*RulesDecisionEngine*” ou pelo “*LLMDecisionEngine*”.

Embora o redirecionamento e a paralelização sejam discutidos no referencial teórico como possibilidades gerais de orquestração dinâmica, essas operações não integram o espaço de ações do experimento reduzido. A topologia composta por apenas dois microserviços não oferece, neste recorte, uma implementação distinta e comparável para essas ações. Assim, o espaço experimental será limitado a CONTINUE, RETRY, WAIT, FALLBACK e ABORT.

No ambiente reduzido, FALLBACK não implicará o encaminhamento para um terceiro microserviço. A ação representará a substituição da rota lógica “*inventory.primary*” por “*inventory.fallback*”, ambas processadas pelo Serviço de Estoque, mantendo uma alternativa operacional controlada sem ampliar a topologia do experimento. Essa ação somente possui significado quando a rota primária estiver degradada ou indisponível de maneira localizada; se o

container inteiro do Serviço de Estoque estiver indisponível, FALLBACK não será ação válida, restando WAIT ou ABORT.

4.4 CENÁRIOS EXPERIMENTAIS

Os cenários experimentais consistirão em perturbações controladas aplicadas ao ambiente descrito na seção Seção 4.3. Em cada cenário, serão mantidos constantes os componentes, os mecanismos de coleta e o fluxo de decisão definido nas subseções Subseção 4.3.2 a Subseção 4.3.12, alterando-se apenas a condição operacional investigada. As perturbações produzirão alterações nos sinais que compõem o “*SYSTEM_STATE*”, permitindo comparar como os dois mecanismos decisórios respondem ao mesmo estado observado.

A simulação desses cenários será fundamentada nos princípios de *Chaos Engineering*, compreendido como a realização de perturbações controladas em um ambiente representativo para avaliar a capacidade do sistema de manter ou recuperar seu comportamento esperado. Conforme Zhang et al. (2024), a avaliação de sistemas autônomicos baseados em LLM pode empregar técnicas de *Chaos Engineering* para introduzir falhas deliberadas em microserviços e observar a capacidade de detecção, reação e mitigação do sistema. De forma complementar, Cornacchia et al. (2025) destacam que a análise de falhas em microserviços exige correlação entre “logs”, métricas, “traces” e estados distribuídos, o que justifica a coleta estruturada de evidências durante os cenários de falha. Esse enquadramento evita que os cenários sejam tratados como simulações *ad hoc*, aproximando-os de uma prática reconhecida de avaliação de confiabilidade em sistemas distribuídos.

No presente estudo, o “*steady state*” será definido como o comportamento esperado do sistema em execução normal, sem falhas induzidas. Esse estado será caracterizado por serviços ativos, respostas válidas, taxa de erro dentro do limite definido no experimento, consumo de recursos estável e fluxo de mensagens processado sem acúmulo anormal. A partir desse comportamento base, serão introduzidas perturbações controladas para observar se a orquestração baseada em regras e a orquestração baseada em agente LLM apresentam diferenças em eficiência, adaptabilidade e resiliência.

A literatura reforça a importância desse tipo de avaliação. Zhang et al. (2024), ao investigar computação autônoma com LLMs em microserviços, empregam técnicas de *Chaos Engineering* para introduzir falhas deliberadas e observar como um sistema de gerenciamento baseado em agentes reage aos problemas. Cornacchia et al. (2025) também destacam que a análise de falhas em microserviços exige correlação entre métricas, “logs”, “traces” e estados distribuídos, o que sobrecarrega ferramentas baseadas em regras e equipes de SRE. Portanto, os cenários deste trabalho são adequados para testar se um agente LLM consegue apoiar decisões de orquestração diante de situações em que regras fixas podem ser insuficientes.

Cada cenário deverá ser executado para as duas abordagens: orquestração baseada

TABELA 12 – Cenários experimentais e forma de operacionalização

Cenário	Sinais alterados
Sobrecarga	queue_size, latency_ms, eventualmente CPU/RAM
Falha intermitente	service_status, last_result, attempt_number
“Atraso”/“timeout”	latency_ms, last_result, attempt_number
Dados inconsistentes	last_result, estado de validação
Recuperação	service_status, queue_size, recent_events

FONTE: Elaborada pelo autor com apoio da ferramenta Chat GPT 5.5, com base no delineamento experimental desta pesquisa.

em regras e orquestração apoiada por agente LLM. A comparação entre as abordagens será feita mantendo a mesma topologia, o mesmo volume de entrada, a mesma janela de coleta, o mesmo tipo de falha e os mesmos critérios de avaliação. Essa simetria é necessária para reduzir interferências externas e permitir que diferenças observadas sejam atribuídas, com maior segurança, ao mecanismo de decisão adotado.

Os cenários de falha distinguirão a degradação da rota primária da indisponibilidade total do Serviço de Estoque. Na degradação ou falha localizada da rota “*inventory.primary*”, as ações possíveis são RETRY, WAIT, FALLBACK e ABORT; na indisponibilidade total do “*inventory-service*”, restam WAIT e ABORT. Nos cenários em que a ação FALLBACK for avaliada, a perturbação será aplicada de forma a degradar a rota primária sem tornar indisponível todo o Serviço de Estoque, pois a rota alternativa pertence ao mesmo microserviço. Cenários de indisponibilidade total do “*inventory-service*” serão tratados separadamente, não sendo FALLBACK considerado uma ação executável nessas condições.

4.4.1 Controle de repetibilidade e reprodutibilidade experimental

Para este trabalho, repetibilidade experimental corresponde à capacidade de executar novamente um mesmo cenário, no mesmo ambiente e sob a mesma configuração, preservando as condições controláveis definidas no delineamento. A reprodutibilidade, por sua vez, refere-se à possibilidade de reconstrução posterior do experimento a partir da documentação, do código, das configurações e dos artefatos registrados. Dessa forma, a metodologia busca assegurar diretamente a repetibilidade das execuções e fornecer elementos suficientes para favorecer sua posterior reprodutibilidade.

Antes da coleta definitiva será realizada uma fase de piloto técnico destinada à validação da comunicação entre componentes, da persistência, dos contratos de mensagem, da idempotência, da construção do estado, da execução das ações, da integração do “*runtime*” LLM e da geração dos artefatos de rastreabilidade. As execuções realizadas nessa fase poderão ser utilizadas para correção de erros e ajuste dos parâmetros ainda não congelados, mas não integrarão a amostra utilizada na análise estatística. Cada execução do piloto será identificada nos metadados por “*phase = PILOT*” e “*eligible_for_sample = false*”; as execuções da coleta definitiva

utilizarão “`phase = EXPERIMENT`” e “`eligible_for_sample = true`”. As execuções de piloto não contarão como N_{rep} e não poderão ser reutilizadas posteriormente como dados definitivos.

Antes de cada repetição, o ambiente experimental será restaurado para um estado inicial previamente definido. O procedimento incluirá a interrupção da execução anterior, a limpeza das filas utilizadas no “*RabbitMQ*”, a remoção das mensagens presentes na “*dead-letter queue*” (DLQ), a restauração dos bancos SQLite dos microserviços para o estado inicial correspondente ao cenário e a remoção de estados transitórios produzidos pela execução anterior. Somente após essa restauração será permitida a inicialização de uma nova repetição.

O início da janela principal de medição dependerá da confirmação de um estado de prontidão do ambiente. Serão considerados critérios mínimos de “*readiness*”: “*containers*” ativos, filas “*RabbitMQ*” declaradas, microserviços respondendo ao mecanismo de “*health check*”, bancos SQLite inicializados no estado esperado, componentes de coleta ativos e, na abordagem baseada em LLM, “*runtime*” disponível, modelo carregado e inferência de aquecimento concluída. O horário de início da medição será registrado somente após a validação desses critérios.

Os cenários serão definidos por arquivos de configuração versionados, contendo os parâmetros necessários para reproduzir a carga e as perturbações aplicadas. Esses arquivos deverão registrar, conforme o cenário: quantidade de tarefas, taxa de envio, “*dataset*” utilizado, “*timeout*”, limite total de tentativas (“*max_attempts*”), intervalos de nova tentativa e de espera, limite de esperas (“*max_waits*”), tipo de falha, rota ou serviço afetado, instante de início da perturbação, duração, intensidade ou probabilidade da falha e sementes aleatórias. O mesmo arquivo de configuração e o mesmo conjunto de tarefas serão utilizados nas execuções “*Rules*” e LLM pertencentes ao mesmo cenário. O Código 11 apresenta um exemplo conceitual da estrutura desse arquivo; os valores exibidos são meramente ilustrativos e deverão ser substituídos pelos parâmetros efetivamente utilizados antes da coleta final.

Considerando a influência dos parâmetros de decodificação sobre a variabilidade da geração discutida na Subseção 3.3.5, modelo, “*prompt*”, “*temperature*”, “*top_p*”, limite de saída e demais parâmetros suportados pelo “*runtime*” serão mantidos constantes durante a coleta definitiva e registrados nos metadados de execução. Os valores concretos desses parâmetros serão definidos somente quando o modelo e o “*runtime*” finais estiverem escolhidos.

As fontes controláveis de variabilidade do agente LLM também serão mantidas constantes entre as repetições. Serão registrados o modelo utilizado, versão ou identificador do modelo, quantização, “*runtime*” e respectiva versão, parâmetros de geração disponibilizados pela ferramenta de inferência, como “*temperature*”, “*top_p*”, limite de “*tokens*” e “*seed*”, quando aplicáveis. Esses controles buscam reduzir a variabilidade decorrente da configuração, sem pressupor respostas idênticas em todas as chamadas, uma vez que o não determinismo do modelo pode permanecer presente mesmo em condições semelhantes. Além de favorecer a repetibilidade, a manutenção desses elementos constantes reduz o risco de confundir alterações de versão ou

Código 11 – Exemplo conceitual de experiment_config.yml

```
1 experiment:
2   repetitions: N_rep
3   execution_order: alternating
4
5 workload:
6   dataset: datasets/orders_v1.json
7   requests: 500
8   rate_per_second: 10
9   seed: 1042
10
11 messaging:
12   inventory_timeout_ms: 2000
13   max_attempts: 3
14   retry_delay_ms: 500
15   wait_delay_ms: 1000
16   max_waits: 2
17   fallback_max_attempts: 1
18   queue_high_watermark: 50
19
20 context:
21   recent_events_limit: 10
22
23 fault:
24   type: intermittent_error
25   target_route: inventory.primary
26   start_after_seconds: 20
27   duration_seconds: 30
28   failure_probability: 0.25
29   seed: 2048
30
31 llm:
32   runtime: ollama
33   runtime_version: "<versao>"
34   model: llama3.1:8b
35   model_digest: "<identificador ou digest>"
36   quantization: "<quantizacao>"
37   temperature: 0.0
38   top_p: 1.0
39   max_tokens: 64
40   seed: 42
41   request_timeout_seconds: 30
42   session_memory: false
43   stream: false
```

FONTE: Elaborada pelo autor com base no delineamento experimental desta pesquisa.

de configuração com mudanças do próprio comportamento decisório do agente ao longo das execuções.

Cada repetição possuirá um “*execution_id*” único e será associada à configuração efetivamente utilizada. Além dos dados brutos e métricas, serão registrados metadados da execução, incluindo versão do código, configuração do cenário, versão das imagens ou componentes executados, características relevantes do “*hardware*” e “*software*”, horário de início e fim e resultado da verificação de “*readiness*”. Essa rastreabilidade permite identificar as condições que produziram cada resultado e facilita a reconstrução do experimento.

Execuções comprometidas por falhas não previstas no cenário, como erro de inicialização dos “*containers*”, indisponibilidade do mecanismo de coleta, corrupção ou ausência de arquivos obrigatórios, encerramento inesperado do “*runtime*” do LLM ou inconsistência detectada no estado inicial, serão classificadas como execuções inválidas. Essas execuções não serão incorporadas silenciosamente ao conjunto analítico: o motivo da invalidação será registrado e uma nova execução será realizada utilizando a mesma configuração experimental.

Para reduzir possíveis efeitos temporais decorrentes de aquecimento de “*hardware*”, “*cache*”, processos residuais ou mudanças progressivas no ambiente local, a ordem das abordagens deverá ser previamente definida e registrada. Recomenda-se a alternância entre “*Rules*” e LLM dentro de cada cenário ou outra ordem previamente fixada, evitando executar todas as repetições de uma abordagem antes de iniciar a outra sem justificativa metodológica.

As sementes aleatórias serão utilizadas para controlar fontes de variabilidade implementadas nos próprios “*scripts*” experimentais. Quando aplicável, serão registradas separadamente a “*seed*” utilizada na geração ou seleção da carga (“*seed_workload*”), a “*seed*” associada à injeção probabilística de falhas (“*seed_fault*”) e a “*seed*” disponibilizada pelo “*runtime*” de inferência do LLM (“*seed_llm*”). O uso dessas sementes busca permitir a repetição das condições programadas, sem pressupor que o LLM produzirá respostas idênticas em todas as execuções.

Para evitar que diferenças entre as abordagens sejam causadas por conjuntos distintos de tarefas, o mesmo “*dataset*” de pedidos sintéticos será utilizado nas execuções “*Rules*” e LLM de um mesmo cenário. O “*dataset*” será versionado e permanecerá inalterado durante a coleta definitiva. Quando houver geração procedural de tarefas, a “*seed*” e os parâmetros utilizados deverão permitir a reconstrução do mesmo conjunto de entrada.

O início da janela de medição dependerá da aprovação de uma verificação automática de “*readiness*”. A execução somente será iniciada quando: (a) todos os “*containers*” necessários estiverem ativos; (b) as filas “*RabbitMQ*” previstas estiverem declaradas; (c) os microserviços estiverem respondendo aos mecanismos de “*health check*”; (d) os bancos SQLite estiverem no estado inicial previsto; (e) os coletores de métricas estiverem ativos; e (f), na abordagem LLM, o modelo estiver carregado e o “*warm-up*” concluído.

A ordem de execução das abordagens será definida previamente e registrada nos metadados. Preferencialmente, serão alternadas execuções “*Rules*” e LLM dentro de cada cenário, de forma a reduzir possíveis efeitos temporais decorrentes de aquecimento de “*hardware*”, “*cache*” ou carga residual do ambiente. Caso seja adotada outra ordem, esta permanecerá fixa e documentada antes da coleta definitiva.

Uma execução será considerada inválida quando ocorrer falha da própria infraestrutura experimental não prevista no cenário, ausência ou corrupção de arquivo obrigatório, falha na inicialização ou no “*readiness*”, encerramento inesperado de componente necessário ou inconsistência detectada no estado inicial. Execuções invalidadas serão registradas com seu motivo e não serão substituídas silenciosamente. Uma nova repetição será realizada utilizando a mesma configuração até completar o número previsto de execuções válidas.

TABELA 13 – Elementos de controle da repetibilidade e reprodutibilidade experimental

Elemento controlado	Estratégia adotada
Código-fonte	Versão/“ <i>commit</i> ” registrado nos metadados
“ <i>Docker Compose</i> ”	Mesmo arquivo e mesmas versões de imagens
RabbitMQ	Filas e DLQ restauradas antes de cada repetição
SQLite	Estado inicial restaurado para cada microserviço
Carga	Mesmo “ <i>dataset</i> ”, volume e taxa para <i>Rules</i> e LLM
Falha induzida	Mesma configuração, duração, intensidade e “ <i>seed</i> ”
<i>Timeout, attempts, wait e fallback</i>	Mesmos valores, limites e intervalos em ambas as abordagens
Contrato de mensagens	Mesma versão do <i>schema</i> e mesma topologia lógica de filas
Validador	Mesmo código e mesmas restrições
Executor	Mesmo código e mesma semântica das ações
<i>Rules</i>	Mesma política e mesmos limiares durante o experimento
LLM	Mesmo modelo, quantização, “ <i>runtime</i> ” e parâmetros de geração
“ <i>Prompt</i> ”	Mesmo “ <i>template</i> ” fixo
“ <i>SYSTEM_STATE</i> ”	Mesmo contrato, mesma política de seleção de contexto, mesmo limite de “ <i>recent_events</i> ” e mesma lógica do “ <i>State Builder</i> ”
“ <i>Warm-up</i> ”	Executado antes da janela principal de medição
Coleta	Mesmo intervalo de amostragem e mesmos arquivos
Número de repetições	N_{rep} definido antes da coleta definitiva
Ordem de execução	Alternada ou previamente estabelecida
Fase da execução	Pilotos excluídos da amostra por metadado explícito (“ <i>phase</i> ”, “ <i>eligible_for_sample</i> ”)
Execução inválida	Motivo registrado e repetição da mesma configuração

FONTE: Elaborada pelo autor com base no delineamento experimental desta pesquisa.

A estrutura de diretórios recomendada para o projeto experimental, contemplando os arquivos de configuração versionados, o “*dataset*” fixo, os dados brutos por execução e os registros de execuções inválidas, é apresentada no Quadro 4. Os artefatos produzidos durante o piloto serão armazenados em “<data/pilot/>”, separadamente daqueles pertencentes à coleta definitiva, em “<data/experiment/>”, o que permite impedir a inclusão acidental de execuções de calibração na amostra.

O protocolo apresentado a seguir aplica-se às execuções integrantes da amostra. A fase de

piloto técnico será realizada anteriormente e seus artefatos serão mantidos separadamente. Após o congelamento dos parâmetros, o arquivo “`experiment_config.yml`” passará a ser a fonte de verdade versionada da configuração. O bloco “messaging” conterá `inventory_timeout_ms`, `max_attempts`, `retry_delay_ms`, `wait_delay_ms`, `max_waits`, `fallback_max_attempts` e `queue_high_watermark`; o bloco “context” conterá `recent_events_limit`; e o bloco “llm” conterá `runtime`, `runtime_version`, `model`, `identificador` ou `model_digest`, `quantization`, `temperature`, `top_p`, `max_tokens`, `seed` (quando suportada), `session_memory` igual a `false`, `request_timeout_seconds` e `stream` igual a `false`. O protocolo experimental será composto pelas seguintes etapas:

TABELA 14 – Protocolo operacional dos experimentos

Etapa	Procedimento	Arquivo ou registro produzido
1	Selecionar o cenário, a abordagem e a repetição e carregar o arquivo de configuração versionado.	<experiment_config.yml>; execution_id; scenario_id; repetition_id
2	Restaurar o estado inicial: limpar filas/DLQ, restaurar SQLite e remover estados transitórios.	<reset.log>; initial_database_hashes; queue_reset_status
3	Inicializar “ <i>Docker Compose</i> ”, microserviços, “ <i>RabbitMQ</i> ”, orquestrador e coleta.	<docker-compose.yml>; “logs” de inicialização
4	Executar verificação automática de “ <i>readiness</i> ”.	readiness_status; readiness_timestamp
5	Para LLM, carregar o modelo e executar “ <i>warm-up</i> ”.	model_load_ms; warmup_inference_ms; <warmup.log>
6	Registrar metadados, estado inicial, versões e configuração.	<execution_metadata.json>; <initial_state.json>
7	Executar carga controlada com “ <i>dataset</i> ” e parâmetros do cenário.	<task_events.jsonl>
8	Induzir falha/atraso/inconsistência/sobrecarga conforme a configuração.	<fault_events.jsonl>
9	Coletar estados, decisões, “logs”, métricas “ <i>RabbitMQ</i> ” e de “ <i>containers</i> ”.	<states.jsonl>; <decisions.jsonl>; <microservices_logs.jsonl>; <queue_metrics.csv>; <container_stats.csv>
10	Encerrar o cenário pelo critério definido e registrar o retorno ou não ao “ <i>steady state</i> ”.	<states.jsonl>; <task_events.jsonl>; <execution_metadata.json>
11	Validar a integridade da execução.	run_validation_status; invalid_reason quando aplicável
12	Se válida, consolidar as métricas; se inválida, registrar o motivo e repetir a mesma configuração.	<metrics_summary.csv>; <invalid_runs.csv>
13	Após N_{rep} execuções válidas por cenário $\times$ abordagem, realizar a análise estatística.	<analysis_summary.md>; <results.csv>

FONTE: Elaborada pelo autor com base em Antonio GIL (2022), Creswell e Creswell (2021), Cornacchia et al. (2025), Zhang et al. (2024), Yu et al. (2025) e Yu, Ding e Sato (2025).

O TCC utiliza “*Python*”, “*Docker Compose*”, “*RabbitMQ*”, “*Prometheus*”, “*Docker Stats*”, “logs” estruturados e o modelo LLM local indicado no delineamento. A injeção de falhas será feita por “*scripts*” próprios em “*Python*” e comandos de controle do ambiente “*Docker*”, o que preserva simplicidade e reprodutibilidade no escopo de um TCC.

Os parâmetros operacionais de timeout, retry e demais limites de segurança serão

previamente definidos e mantidos constantes entre as abordagens, conforme estabelecido na seção 4.3.

O tempo de carregamento do modelo (`model_load_ms`) e o tempo da inferência de aquecimento (`warmup_inference_ms`) integram os metadados da execução, mas não devem ser somados ao `decision_time_ms` utilizado para comparar decisões em regime estabilizado.

4.5 MÉTRICAS E COLETA DE DADOS

Esta seção apresenta os indicadores utilizados para avaliar os resultados produzidos pelo fluxo experimental definido na Seção 4.3. Os sinais utilizados online pelo mecanismo decisório são especificados na Subseção 4.3.4; aqui são descritas as métricas calculadas para análise comparativa após ou durante as execuções.

As métricas utilizadas neste trabalho serão organizadas de modo a distinguir o custo diretamente associado ao mecanismo de decisão dos efeitos produzidos por suas ações sobre o sistema. Essa separação é necessária porque uma abordagem baseada em LLM pode introduzir maior tempo de decisão e maior consumo computacional e, ainda assim, produzir uma redução no tempo total de recuperação ou na quantidade de falhas. Inversamente, o custo de inferência pode não ser compensado por ganhos operacionais. Por esse motivo, a análise não tratará a latência da inferência e a latência ponta a ponta como medidas equivalentes.

Neste trabalho, denomina-se **overhead decisório** o custo incremental associado à substituição do motor baseado em regras pelo agente LLM para produzir uma ação válida, mantendo-se equivalentes o estado operacional, o mecanismo de validação e o executor. O overhead será observado por meio do tempo de decisão, tempo acumulado de inferência, número de chamadas ao modelo, volume de “tokens” quando disponível e consumo computacional. O **impacto sistêmico líquido**, por sua vez, será avaliado por métricas ponta a ponta, como latência, “throughput”, taxa de erro, tempo de recuperação e comportamento das filas. Essa distinção permite avaliar simultaneamente o custo e o benefício operacional da abordagem baseada em LLM (CORNACCHIA et al., 2025; AMAYUELAS et al., 2025; MIAO et al., 2023; LUO et al., 2025).

4.5.1 Notação e formalização matemática

Para formalizar as métricas de overhead e impacto sistêmico, considere: R como a abordagem baseada em regras; L como a abordagem baseada em LLM; s como o cenário experimental; i como a repetição; e k como o ponto de decisão dentro de uma execução.

O tempo de decisão de um ponto k é definido como

$$T_{dec,k} = t_{acao_pronta,k} - t_{estado_pronto,k} \quad (4.1)$$

em que t_{estado_pronto} é o instante em que o “SYSTEM_STATE” está disponível ao motor decisório, e t_{acao_pronta} é o instante em que uma ação válida está disponível para execução.

Para cada execução, o tempo decisório acumulado é

$$T_{dec,total} = \sum_{k=1}^{N_{dec}} T_{dec,k} \quad (4.2)$$

e, para a abordagem LLM, o tempo acumulado de inferência é

$$T_{inf,total} = \sum_{k=1}^{N_{LLM}} T_{inf,k} \quad (4.3)$$

O tempo médio por decisão,

$$T_{dec,medio} = \frac{T_{dec,total}}{N_{dec}} \quad (4.4)$$

é uma medida diagnóstica. Para o overhead, o valor acumulado por execução é particularmente importante, pois as abordagens podem gerar quantidades diferentes de decisões.

O overhead decisório absoluto por cenário é definido como

$$OH_{dec}(s) = \overline{T_{dec,total}}(L, s) - \overline{T_{dec,total}}(R, s) \quad (4.5)$$

em que $OH_{dec} > 0$ indica que a camada LLM acrescentou tempo ao processo decisório; $OH_{dec} = 0$ indica ausência de diferença observável no tempo decisório acumulado; e $OH_{dec} < 0$ representa uma situação improvável para inferência local convencional, mas matematicamente possível caso a arquitetura adotada reduza outras etapas do caminho decisório.

O overhead decisório relativo é dado por

$$OH_{dec\%}(s) = \frac{\overline{T_{dec,total}}(L, s) - \overline{T_{dec,total}}(R, s)}{\overline{T_{dec,total}}(R, s)} \times 100 \quad (4.6)$$

Como regras determinísticas podem decidir em tempos muito próximos de zero, o overhead percentual pode assumir valores extremamente elevados e pouco intuitivos. Por isso, este trabalho trata a diferença absoluta em milissegundos (Equação 4.5) como medida principal, e o percentual (Equação 4.6) como informação complementar.

O número de chamadas ao LLM por tarefa e o volume de “tokens” são calculados, respectivamente, como

$$Calls_{task} = \frac{N_{LLM_calls}}{N_{tasks}} \quad (4.7)$$

$$Tokens_{total} = \sum (input_tokens + output_tokens) \quad (4.8)$$

$$Tokens_{task} = \frac{Tokens_{total}}{N_{completed_tasks}} \quad (4.9)$$

Essas métricas são auxiliares e ajudam a explicar variações no tempo de inferência.

O impacto sistêmico sobre a latência ponta a ponta e sobre o “*throughput*” é definido, respectivamente, como

$$\Delta_{E2E}(s) = \overline{T_{E2E}}(L, s) - \overline{T_{E2E}}(R, s), \quad \Delta_{Thr}(s) = \overline{Thr}(L, s) - \overline{Thr}(R, s) \quad (4.10)$$

Para latência e tempo de recuperação, $\Delta < 0$ favorece a abordagem LLM e $\Delta > 0$ favorece a linha de base baseada em regras; para “*throughput*”, $\Delta > 0$ favorece LLM e $\Delta < 0$ favorece *Rules*. A expressão “impacto sistêmico” deve ser usada exclusivamente para essas diferenças ponta a ponta: Δ_{E2E} não deve ser confundido com overhead, pois uma decisão LLM pode possuir overhead decisório positivo e, simultaneamente, reduzir a latência total do fluxo.

4.5.2 Decomposição conceitual do tempo de decisão

Para fins de interpretação, o tempo de decisão de cada abordagem pode ser conceitualmente decomposto como

$$T_{dec}(R) \approx T_{rules} + T_{validation}, \quad T_{dec}(L) \approx T_{prompt} + T_{inference} + T_{parsing} + T_{validation} \quad (4.11)$$

O “*State Builder*” não entra nessa decomposição do overhead decisório por ser compartilhado pelas duas abordagens. A metodologia não instrumenta individualmente T_{prompt} , $T_{parsing}$ e $T_{validation}$: a decomposição é conceitual, mantendo-se como medidas obrigatórias apenas “*decision_time_ms*” e “*llm_inference_ms*”.

Quadro 4 – Estrutura de diretórios recomendada para o projeto experimental

```
project/
|-- docker-compose.yml
|-- config/
|   |-- experiment_config.yml
|   '-- scenarios/
|       |-- normal.yml
|       |-- overload.yml
|       |-- intermittent_failure.yml
|       '-- timeout.yml
|-- datasets/
|   '-- orders_v1.json
|-- data/
|   |-- pilot/
|   |   '-- PILOT_0001/
|   |       |-- execution_metadata.json
|   |       |-- initial_state.json
|   |       |-- states.jsonl
|   |       |-- task_events.jsonl
|   |       |-- decisions.jsonl
|   |       |-- fault_events.jsonl
|   |       |-- microservices_logs.jsonl
|   |       |-- queue_metrics.csv
|   |       '-- container_stats.csv
|   |-- experiment/
|   |   '-- EXP_0042/
|   |       |-- execution_metadata.json
|   |       |-- initial_state.json
|   |       |-- states.jsonl
|   |       |-- task_events.jsonl
|   |       |-- decisions.jsonl
|   |       |-- fault_events.jsonl
|   |       |-- microservices_logs.jsonl
|   |       |-- queue_metrics.csv
|   |       '-- container_stats.csv
|   '-- processed/
|       '-- metrics_summary.csv
|-- logs/
|   |-- reset.log
|   '-- warmup.log
|-- reports/
|   |-- results.csv
|   '-- analysis_summary.md
'-- invalid_runs.csv
```

FONTE: FONTE: Elaborada pelo autor com base no delineamento experimental desta pesquisa.

TABELA 15 – Métricas, justificativa e forma de coleta

Métrica	O que mede	Por que será observada	Como será coletada	Arquivo de saída
Latência média	Tempo médio entre entrada da tarefa e conclusão do fluxo.	Mede rapidez geral de resposta.	“ <i>Timestamps</i> ” registrados pelo orquestrador e microserviços em “ <i>logs</i> ” estruturados.	<latency_metrics.csv>
Latência P95	Tempo abaixo do qual 95% das execuções foram concluídas.	Captura degradação percebida em piores casos, mais relevante que apenas média.	Cálculo a partir dos tempos individuais de execução.	<latency_metrics.csv>
“ <i>Throughput</i> ”	Quantidade de tarefas concluídas por unidade de tempo.	Mede capacidade de processamento sob carga.	Contagem de mensagens/requisições concluídas dividida pela duração da execução.	<throughput_metrics.csv>
Taxa de erro	Proporção de tarefas com erro, “ <i>timeout</i> ” ou resposta inválida.	Mede confiabilidade operacional.	“ <i>Logs</i> ” dos serviços e do orquestrador, com “ <i>status</i> ” de cada tarefa.	<error_metrics.csv>
Tempo de recuperação	Tempo entre a falha induzida e o retorno ao estado estável.	Mede resiliência e capacidade de retomada.	Comparação entre “ <i>timestamp</i> ” da falha e “ <i>timestamp</i> ” de normalização dos indicadores.	<recovery_metrics.csv>
Uso de CPU	Consumo de CPU dos “ <i>containers</i> ” durante cada cenário, registrado ao menos como média e pico por execução.	Indica custo computacional da abordagem.	<code>docker stats</code> e/ou métricas exportadas ao “ <i>Prometheus</i> ”, preferencialmente distinguindo o processo ou “ <i>container</i> ” responsável pela inferência.	<container_stats.csv>
Continua na próxima página. . .				

TABELA 15 – Continuação

Métrica	O que mede	Por que será observada	Como será coletada	Arquivo de saída
Uso de memória	Consumo de RAM dos “ <i>containers</i> ”, registrado ao menos como média e pico por execução.	Indica custo operacional e viabilidade local.	<code>docker stats</code> e/ou “ <i>Prometheus</i> ”, preferencialmente distinguindo o processo ou “ <i>container</i> ” responsável pela inferência.	<container_stats.csv>
Tamanho da fila	Quantidade de mensagens pendentes no “ <i>broker</i> ”.	Mede acúmulo causado por lentidão, falha ou sobrecarga.	Métricas do “ <i>RabbitMQ</i> ” e “ <i>logs</i> ” de produtores/consumidores.	<queue_metrics.csv>
Tempo de decisão do orquestrador	Tempo entre a disponibilização do estado operacional e a obtenção de uma ação válida.	Permite comparar o custo temporal do mecanismo decisório entre <i>Rules</i> e LLM.	“ <i>Timestamps</i> ” registrados em <decisions.jsonl> para ambas as abordagens.	<decisions.jsonl>
Tempo de inferência do LLM	Parcela do tempo decisório correspondente à chamada ao modelo.	Isola o custo específico da inferência dentro do tempo total de decisão.	“ <i>Timestamp</i> ” imediatamente antes e após a chamada ao “ <i>runtime</i> ” do modelo.	<decisions.jsonl>
Quantidade de decisões/chamadas ao LLM	Número de pontos de decisão e número de inferências realizadas em cada execução.	Permite identificar acúmulo de overhead em trajetórias com múltiplas decisões.	Contagem dos registros em <decisions.jsonl> por <code>execution_id</code> e <code>decision_engine</code> .	<decisions.jsonl>
Tokens de entrada e saída	Volume de contexto processado e de resposta gerada pelo modelo.	Auxilia a explicar variações de latência e custo de inferência.	Contagem fornecida pelo “ <i>runtime</i> ” ou “ <i>tokenizer</i> ” utilizado, quando disponível de forma confiável.	<decisions.jsonl>
Continua na próxima página. . .				

TABELA 15 – Continuação

Métrica	O que mede	Por que será observada	Como será coletada	Arquivo de saída
Overhead decisório absoluto	Diferença entre o tempo decisório acumulado da abordagem LLM e da abordagem <i>Rules</i> sob o mesmo cenário.	Quantifica diretamente o custo temporal adicional da camada LLM.	Calculado posteriormente a partir de <decisions.jsonl>, conforme Subseção 4.5.1.	<results_summary.csv>
Tipo de decisão do orquestrador	Ação escolhida: CONTINUE, RETRY, WAIT, FALLBACK ou ABORT.	Permite comparar comportamento decisório entre regras e LLM.	“Logs” estruturados do orquestrador.	<decisions.jsonl>
“Blast radius”	Quantidade de serviços, tarefas ou mensagens afetadas por uma falha.	Mede propagação do impacto da falha no sistema.	Correlação entre “logs” de erro, serviços afetados e tarefas interrompidas.	<blast_radius.csv>

FONTE: Elaborada pelo autor com base em Cornacchia et al. (2025), Pei et al. (2025), Yu, Ding e Sato (2025), Ardebili e Bartolini (2025), Zhang et al. (2024) e na pesquisa dirigida para reforço bibliográfico do TCC.

Opcionalmente, caso a frequência de amostragem seja estável, o consumo de CPU e de memória poderá também ser expresso como uma medida acumulada, aproximada pela área sob a curva:

$$CPU_{AUC} \approx \sum_j CPU_j \times \Delta t_j, \quad RAM_{AUC} \approx \sum_j RAM_j \times \Delta t_j \quad (4.12)$$

com unidades aproximadas de %CPU·s e MB·s, respectivamente. Essas medidas não são necessárias para validar o experimento, mas são superiores à média isolada quando as execuções possuem durações diferentes.

4.5.3 Instrumentos de Coleta de Dados

Os instrumentos de coleta utilizarão os identificadores e a estrutura de rastreabilidade definidos na Subseção 4.3.10. Dessa forma, estados observados, eventos das tarefas, decisões, ações executadas e métricas operacionais poderão ser correlacionados por execução e por tarefa. Além dos arquivos de dados brutos, cada execução produzirá um arquivo `<execution_metadata.json>` contendo informações suficientes para identificar a configuração efetivamente utilizada. Esse arquivo registrará, no mínimo, `execution_id`, `phase`, `eligible_for_sample`, `scenario_id`, `repetition_id`, abordagem, “*seed*” ou sementes aplicáveis, “*dataset*” utilizado, versão do código, versão do “*Docker Compose*”, versões dos componentes relevantes, modelo LLM, identificador ou *digest* do modelo, quantização, “*runtime*” e versão, parâmetros de geração, “*hardware*” relevante, horário de início e fim, `model_load_ms`, `warmup_inference_ms`, resultado da verificação de “*readiness*”, “`run_status`” e “`invalid_reason`” quando aplicável.

Quando possível, arquivos críticos de configuração e “*datasets*” também poderão ter seu “*hash*” registrado nos metadados. Essa medida permitirá verificar posteriormente se duas execuções utilizaram exatamente os mesmos artefatos, reduzindo ambiguidades relacionadas à alteração acidental de arquivos entre repetições. O Código 12 apresenta os campos recomendados para `<execution_metadata.json>`; campos não disponíveis não devem ser inventados, podendo ser removidos ou registrados como `null`.

O campo “`context_policy`” registra a política de seleção de contexto efetivamente aplicada na execução, incluindo o limite de “`recent_events`” (“`recent_events_limit`”) e a indicação de que o “*runtime*” não manterá memória de sessão entre chamadas (“`llm_session_memory`”); esse último campo somente deve ser incluído se o comportamento do “*runtime*” puder ser controlado e confirmado. O campo “`message_ordering_policy`” registra opcionalmente que a ordenação controlada é definida por tarefa (scope: `per_task`), representada pelo campo “`event_seq`”, e que não são utilizados ordenação total global (`global_total_order`) nem relógio lógico (`logical_clock`).

Código 12 – Campos recomendados para execution_metadata.json

```
1 {
2   "execution_id": "EXP_0042",
3   "phase": "EXPERIMENT",
4   "eligible_for_sample": true,
5   "scenario_id": "timeout",
6   "repetition_id": 7,
7   "decision_engine": "LLM",
8   "experiment_version": "1.0",
9   "git_commit": "<commit>",
10  "scenario_config": "config/scenarios/timeout.yml",
11  "scenario_config_hash": "<hash>",
12  "dataset": "datasets/orders_v1.json",
13  "dataset_hash": "<hash>",
14  "seeds": {
15    "workload": 1042,
16    "fault": 2048,
17    "llm": 42
18  },
19  "software": {
20    "python_version": "<versao>",
21    "docker_compose_version": "<versao>",
22    "rabbitmq_version": "<versao>",
23    "llm_runtime": "<runtime>",
24    "llm_runtime_version": "<versao>"
25  },
26  "llm": {
27    "model": "llama3.1:8b",
28    "model_digest": "<identificador ou digest>",
29    "quantization": "<quantizacao>",
30    "temperature": 0.0,
31    "top_p": 1.0,
32    "max_tokens": 64,
33    "seed": 42,
34    "request_timeout_seconds": 30,
35    "session_memory": false,
36    "stream": false
37  },
38  "hardware": {
39    "cpu": "<cpu>",
40    "ram_gb": "<ram>",
41    "gpu": "<gpu ou null>"
42  },
43  "context_policy": {
44    "recent_events_limit": "<K>",
45    "llm_session_memory": false
46  },
47  "message_ordering_policy": {
48    "scope": "per_task",
49    "field": "event_seq",
50    "global_total_order": false,
51    "logical_clock": false
52  },
53  "readiness_status": "PASS",
54  "model_load_ms": 0,
55  "..."
```

TABELA 16 – Instrumentos de coleta de dados e arquivos gerados

Instrumento/registo	Conteúdo principal	Arquivo
Estado operacional	state_id, “timestamp”, “snapshot” integral do “SYSTEM_STATE” fornecido ao decisor e “recent_events” efetivamente incluídos	<states.jsonl>
Eventos das tarefas	execution_id, task_id, event_id, message_id, event_seq, event_type, serviço, target (“inventory.primary”/“inventory.fallback”) quando registrado, “timestamp”, redelivered, attempt_number; histórico completo da trajetória operacional disponível para análise	<task_events.jsonl>
Decisões do orquestrador	decision_engine, state_id, decisão proposta, validação, ação executada, decision_time_ms, llm_inference_ms quando aplicável, “tokens” quando disponíveis	<decisions.jsonl>
Logs internos dos serviços	eventos internos e erros	<microservices_logs.jsonl>
RabbitMQ	fila, reentrega, rejeição, DLQ	<queue_metrics.csv>
Containers	CPU, RAM, I/O	<container_stats.csv>
Falhas injetadas	tipo, início, duração, fim	<fault_events.jsonl>
Metadados da execução	execution_id, phase, eligible_for_sample, scenario_id, repetition_id, decision_engine, “seeds”, “dataset”, versão/“commit” do código, versões de “software” e imagens, modelo LLM, identificador/digest do modelo, quantização, “runtime” e versão, parâmetros de geração, “hardware” relevante, model_load_ms, warmup_inference_ms, readiness_status, “timestamps” de início/fim, run_status e invalid_reason quando aplicável	<execution_metadata.json>
Execuções inválidas	execution_id, cenário, abordagem, motivo da invalidação, “timestamp” e indicação de repetição	<invalid_runs.csv>

FONTE: Elaborada pelo autor com base em Creswell e Creswell (2021), Antonio GIL (2022), Cornacchia et al. (2025), Pei et al. (2025), Yu, Ding e Sato (2025) e Ardebili e Bartolini (2025).

4.5.4 Instrumentos de Análise dos Dados

A análise dos dados será realizada por meio de estatística descritiva e comparação entre as abordagens de orquestração. Inicialmente, serão calculadas medidas como média, mediana, desvio padrão, mínimo, máximo e percentis, especialmente P95 para latência. Em seguida, os resultados serão agrupados por cenário e por abordagem, permitindo observar diferenças entre orquestração baseada em regras e orquestração baseada em LLM.

A análise do custo da camada LLM será realizada separadamente da análise de seus efeitos sobre o sistema. Para cada cenário, serão calculados o tempo decisório acumulado, o tempo médio por decisão, o tempo total de inferência e a quantidade de chamadas realizadas. Quando a contagem de “tokens” estiver disponível, também serão calculados o total processado

por execução e a quantidade média de “*tokens*” por tarefa concluída. O overhead decisório será expresso prioritariamente pela diferença absoluta, em milissegundos, entre o tempo acumulado de decisão da abordagem baseada em LLM e o da linha de base baseada em regras. Diferenças relativas poderão ser apresentadas como informação complementar, considerando que o baixo tempo de execução das regras pode produzir percentuais excessivamente elevados.

Em paralelo, serão calculadas diferenças entre as abordagens nas métricas ponta a ponta, como latência, “*throughput*”, taxa de erro e tempo de recuperação. Essa separação permitirá identificar situações nas quais o LLM apresenta overhead decisório, mas produz benefício sistêmico por selecionar uma ação mais eficiente, bem como situações em que o custo adicional da inferência não é compensado por melhoria operacional. Dessa forma, os resultados serão interpretados como um “*trade-off*” entre custo decisório e benefício operacional, sem combinar as diferentes métricas em um único índice ponderado.

Para qualquer métrica Y , as diferenças por cenário serão calculadas como

$$\Delta_Y(s) = \bar{Y}(L, s) - \bar{Y}(R, s) \quad (4.13)$$

aplicada por cenário experimental, preservando as distribuições originais para os testes estatísticos. Não será criado um “*score*” geral que combine latência, “*throughput*”, erros, CPU e memória com pesos arbitrários: a interpretação custo-benefício manterá as dimensões separadas.

TABELA 17 – Métodos de análise por métrica

Métrica	Método de análise	Interpretação esperada
Latência média e P95	Comparação por cenário e abordagem; média, mediana e percentis.	Menores valores indicam resposta mais eficiente.
“Throughput”	Comparação do total de tarefas concluídas por tempo.	Maior “throughput” indica maior capacidade de processamento.
Taxa de erro	Percentual de falhas sobre o total de execuções.	Menor taxa indica maior confiabilidade.
Tempo de recuperação	Diferença entre início da falha e retorno ao estado estável.	Menor tempo indica maior resiliência.
CPU e memória	Média, pico e variação durante cada cenário.	Menor consumo, sem perda de desempenho, indica maior eficiência operacional.
Tamanho da fila	Evolução temporal de mensagens pendentes.	Acúmulo persistente indica gargalo ou falha de processamento.
Decisão do orquestrador	comparar decisões produzidas por Rules e LLM diante de estados equivalentes; classificar validade, oportunidade e resultado.	identificar diferenças de comportamento entre os mecanismos decisórios.
Tempo de decisão do orquestrador	Média, mediana, P95 e distribuição por abordagem e cenário.	Quantificar a velocidade do mecanismo decisório.
Overhead decisório	Diferença absoluta entre o tempo decisório acumulado de LLM e de Rules; diferença relativa apenas como complemento.	Quantificar o custo temporal incremental da camada LLM.
Tempo acumulado de inferência	Soma de llm_inference_ms por execução e cenário.	Quantificar quanto do tempo da execução foi consumido diretamente por inferência.
Chamadas/“tokens”	Total por execução e por tarefa concluída.	Explicar intensidade de uso do modelo e possíveis relações com a latência.
Impacto sistêmico	Diferença entre LLM e Rules em latência ponta a ponta, “throughput”, taxa de erro e tempo de recuperação.	Verificar se o custo do LLM é acompanhado por ganho operacional.
“Blast radius”	Contagem de serviços/tarefas impactados após a falha.	Menor propagação indica melhor isolamento de falhas.

FONTE: Elaborada pelo autor com base em Creswell e Creswell (2021), Antonio GIL (2022), Cornacchia et al. (2025), Pei et al. (2025), Yu, Ding e Sato (2025) e Zhang et al. (2024).

A análise quantitativa será complementada por interpretação qualitativa dos “logs” de de:isão. Essa etapa é necessária porque a abordagem baseada em LLM pode apresentar respostas tecnicamente válidas, mas operacionalmente inadequadas, ou respostas que aparentam coerência textual sem corresponder ao estado real do sistema. Cornacchia et al. (2025) alertam que agentes LLM em RCA podem apresentar raciocínios alucinatórios, uso ineficiente de contexto e ações indesejadas. Pei et al. (2025), por sua vez, indicam que o uso de procedimentos estruturados reduz esses riscos em tarefas de diagnóstico. Por isso, as decisões do agente deverão ser analisadas não apenas pelo resultado final, mas também pelo caminho decisório registrado nos “logs”.

TABELA 18 – Técnicas estatísticas e comparativas de análise.

Instrumento de análise	Aplicação
Estatística descritiva	Resumir desempenho de cada abordagem;
Comparação de médias/medianas	Verificar diferenças entre abordagens;
Teste <i>t</i>	Comparar médias quando aplicável;
Teste de Mann-Whitney	Comparar distribuições não normais;
Intervalo de confiança	Estimar margem de variação;
Tamanho de efeito	Medir relevância prática da diferença;
Análise estratificada por cenário	Identificar em quais condições o LLM melhora ou piora.
Correlação exploratória (opcional)	Verificar associação entre total de “tokens” ou número de chamadas e tempo de inferência/tempo decisório, utilizando Pearson quando os pressupostos forem adequados ou Spearman em relação monotônica ou distribuição não paramétrica; item complementar, que não amplia o escopo caso o número de amostras seja limitado.

FONTE: Elaborada pelo autor com apoio da ferramenta Chat GPT 5.5, com base em Antonio GIL (2022), Creswell e Creswell (2021), Yu, Ding e Sato (2025), Amayuelas et al. (2025).

A análise qualitativa das decisões será realizada por meio da correlação entre o estado operacional apresentado ao mecanismo decisório, a decisão inicialmente proposta, o resultado da validação, a ação efetivamente executada e o estado observado posteriormente. Essa sequência permitirá avaliar não apenas se o fluxo foi concluído, mas também se a decisão foi coerente com as informações disponíveis naquele instante e quais efeitos produziu sobre a execução.

De forma exploratória, quando houver quantidade suficiente de ocorrências de estados operacionalmente equivalentes, as decisões do agente também poderão ser observadas segundo a ordem das repetições. O objetivo será verificar se eventuais diferenças aparecem apenas como variação pontual entre execuções ou se apresentam alteração temporal persistente na frequência das ações. Diferenças isoladas não serão classificadas como drift. Qualquer interpretação de mudança comportamental deverá primeiro excluir alterações de configuração, versão, “runtime”, “prompt” ou “SYSTEM_STATE” registradas nos metadados. Esse procedimento permanece exploratório e não será utilizado como critério central para aceitar ou rejeitar a hipótese principal do TCC.

Quando uma decisão LLM for considerada inadequada ao estado operacional, a análise qualitativa deverá verificar também se as informações necessárias para uma decisão alternativa estavam efetivamente presentes no “SYSTEM_STATE” fornecido. Essa distinção permitirá separar erros atribuíveis ao mecanismo decisório de situações em que o próprio recorte de contexto omitiu evidências relevantes.

4.6 LIMITAÇÕES METODOLÓGICAS

A principal limitação metodológica deste trabalho está no uso de uma arquitetura simulada com dois microserviços, executada em ambiente local por meio de “*Docker Compose*”. Essa delimitação é adequada ao escopo de uma pesquisa experimental, pois favorece o controle das variáveis, a repetição dos cenários e a comparação entre abordagens sob condições equivalentes. Contudo, os resultados não devem ser interpretados como generalização direta para ambientes produtivos de larga escala, nos quais há maior número de serviços, dependências, usuários, filas, bases de dados, políticas de segurança e camadas de observabilidade. Essa limitação é coerente com a própria natureza dos microserviços, pois a literatura aponta que arquiteturas com múltiplos serviços ampliam a complexidade operacional, especialmente em cenários de comunicação distribuída, falhas parciais, monitoramento e diagnóstico (NEWMAN, 2015; RICHARDSON, 2019; CORNACCHIA et al., 2025).

A escolha por dois microserviços também representa uma simplificação deliberada. No contexto deste trabalho, essa quantidade permite representar dependência entre serviços, comunicação assíncrona, falha intermediária, reexecução, *fallback* e recuperação, sem ampliar excessivamente a complexidade da implementação. Entretanto, sistemas reais podem envolver dezenas ou centenas de serviços, diferentes bancos de dados, múltiplos brokers, políticas de autenticação, regras de roteamento, balanceamento de carga e infraestrutura distribuída. Portanto, a arquitetura simulada deve ser compreendida como um recorte experimental controlado, e não como reprodução integral de um ambiente corporativo “*cloud-native*”. O próprio TCC já delimita o ambiente como uma simulação local com “*Docker Compose*” e reconhece que dois microserviços buscam equilibrar simplicidade e representatividade.

Outra limitação refere-se à forma de injeção de falhas. Embora os cenários sejam fundamentados em princípios de *Chaos Engineering*, o experimento não utilizará plataformas especializadas como “*Chaos Mesh*” ou “*LitmusChaos*”, pois essas ferramentas não foram integradas ao escopo atual do TCC. A literatura mostra que benchmarks recentes de agentes LLM para incidentes em cloud podem utilizar ferramentas como “*Prometheus*” e “*ChaosMesh*” para observabilidade e injeção de falhas; entretanto, a adoção dessas ferramentas ampliaria a complexidade técnica e introduziria novos componentes não previstos no delineamento atual (ARDEBILI; BARTOLINI, 2025). Por isso, neste trabalho, a perturbação será realizada por “*scripts*” “*Python*” e controle do ambiente “*Docker Compose*”, mantendo aderência às ferramentas já previstas no projeto. Essa decisão favorece a viabilidade e a reprodutibilidade, mas limita a

diversidade e a profundidade dos tipos de falha simulados.

A ausência de uma plataforma específica de *Chaos Engineering* também limita a avaliação do “*blast radius*”, da propagação de falhas e de comportamentos emergentes mais complexos. Zhang et al. (2024) utilizam técnicas de *Chaos Engineering* para introduzir falhas deliberadas em microserviços e observar a reação de um sistema de gerenciamento baseado em LLM, o que reforça a pertinência desse tipo de avaliação. Contudo, como o presente trabalho emprega uma arquitetura reduzida e “*scripts*” próprios de falha, os resultados devem ser interpretados como indícios de comportamento comparativo entre abordagens, e não como validação completa de resiliência em produção.

Também há limitações relacionadas ao agente LLM. Modelos de linguagem podem produzir respostas diferentes diante de entradas semelhantes, dependendo do contexto, da configuração de inferência e das probabilidades internas do modelo. Além disso, a literatura aponta riscos como raciocínio alucinado, uso ineficiente de contexto, chamadas inadequadas de ferramentas e dificuldade de compreender relações causais em sistemas distribuídos (CORNACCHIA et al., 2025; PEI et al., 2025). Esses riscos são especialmente relevantes em tarefas de análise de falhas, nas quais o agente precisa interpretar métricas, “*logs*”, estados de serviços e possíveis relações de causalidade. O não determinismo e as possíveis decisões inválidas do LLM podem introduzir variabilidade adicional nos resultados, mesmo sob estados operacionais semelhantes. Os mecanismos de rastreabilidade e validação definidos nas seções 4.3.9 e 4.3.10 permitem observar essa variabilidade, mas não a eliminam.

Os controles de repetibilidade adotados, descritos na Subseção 4.4.1, reduzem fontes de variabilidade introduzidas pela configuração do ambiente, pelos “*scripts*” de carga e falha e pelos parâmetros de inferência, mas não eliminam completamente a variabilidade do agente LLM. Mesmo com modelo, quantização, “*runtime*”, “*prompt*”, “*SYSTEM_STATE*” e parâmetros de geração mantidos constantes, podem ocorrer diferenças entre respostas produzidas em execuções distintas. Por esse motivo, a repetibilidade deste estudo não será definida como produção de decisões idênticas em todas as chamadas, mas como execução do mesmo protocolo sob condições controladas e análise da distribuição dos resultados obtidos.

O estudo também não tem como objetivo provocar ou mensurar formalmente drift comportamental de longo prazo. O modelo permanecerá congelado durante a coleta, sem aprendizagem online, atualização de pesos ou substituição de versão, e o contexto fornecido ao agente será delimitado pelo “*SYSTEM_STATE*” e pelo histórico recente definido na arquitetura experimental. Essas condições reduzem algumas fontes conhecidas de alteração temporal, mas não permitem generalizar que sistemas agentivos permaneçam estáveis em implantações prolongadas. Caso sejam observadas mudanças sistemáticas nas decisões ao longo das repetições, elas serão tratadas como evidência exploratória e interpretadas à luz dos metadados da execução, e não como medição formal de drift em produção.

Outra limitação está na dependência da instrumentação. A validade das métricas depende

da correta instrumentação e sincronização dos registros descritos em 4.3.10 e 4.5.1. Erros nesses mecanismos podem comprometer a correlação temporal entre estado, decisão e resultado. *KubeIntellect* também reforça a importância de “*pipelines*” de dados, “*logs*”, métricas, “*health probes*” e telemetria para confiabilidade, depuração e avaliação de desempenho em sistemas orquestrados por LLM (ARDEBILI; BARTOLINI, 2025).

Outra limitação refere-se à dependência do overhead da abordagem LLM em relação à configuração de execução. O tempo de inferência, o uso de memória e o consumo de CPU podem variar conforme o modelo selecionado, seu nível de quantização, o “*runtime*” utilizado e as características do “*hardware*” local. Por esse motivo, os valores absolutos obtidos no experimento não devem ser generalizados como custo universal de agentes LLM. O estudo buscará controlar essa influência mantendo a mesma configuração durante todas as repetições, registrando os principais parâmetros do modelo em `<execution_metadata.json>` e separando o carregamento inicial do modelo da janela regular de medição. Os resultados de overhead devem, portanto, ser interpretados como válidos para a configuração experimental documentada.

Embora a literatura indique que a inferência de LLM também possui custo energético (NIU et al., 2025), este trabalho não realizará medição direta de energia, pois isso exigiria instrumentação adicional e ampliaria o escopo experimental. CPU, memória, latência e tempo de inferência serão utilizados como indicadores computacionais do custo da abordagem, sem inferir consumo energético a partir dessas medidas.

Outra limitação decorre do recorte de contexto apresentado ao mecanismo decisório. Embora o histórico completo de eventos e decisões seja preservado para rastreabilidade, o “*SYSTEM_STATE*” contém somente as variáveis e a janela de eventos previamente definidas pelo “*State Builder*”. Essa estratégia favorece comparabilidade e controle do volume de contexto, mas pode excluir informações históricas que, em situações mais complexas, seriam relevantes à decisão. Portanto, os resultados do agente devem ser interpretados em função da representação operacional efetivamente disponibilizada, e não como avaliação de sua capacidade diante de toda a telemetria existente no ambiente.

O experimento também não implementará um mecanismo completo de “*durable execution*” ou “*checkpointing*” do orquestrador. A persistência dos bancos SQLite e dos arquivos de estados, eventos e decisões permite preservar dados de negócio e reconstruir posteriormente a trajetória para fins de análise, mas não implica retomada automática de uma tarefa interrompida exatamente do ponto em que foi suspensa. *KubeIntellect*, por exemplo, utiliza memória efêmera combinada a “*checkpoints*” persistentes para permitir retomada de “*workflows*” após interrupções (ARDEBILI; BARTOLINI, 2025). Tal capacidade está fora do escopo desta pesquisa e deve ser considerada em eventuais aplicações produtivas.

Além disso, o “*SYSTEM_STATE*” representa um “*snapshot*” do ambiente em determinado instante. Como existe intervalo entre a construção do estado e a execução da ação, especialmente na abordagem LLM, alterações ocorridas durante a inferência podem tornar parte das informações

defasadas. Os “*timestamps*” registrados permitirão identificar essa defasagem, mas o trabalho não pretende caracterizar formalmente consistência temporal ou executar protocolos distribuídos de “*snapshot*”.

O ambiente experimental também não implementará mecanismos formais de relógio lógico, consenso distribuído ou tolerância a falhas bizantinas. Os “*timestamps*” serão utilizados para medição e correlação em ambiente local, enquanto a ordem lógica relevante será representada no nível de cada tarefa. Como será utilizado um único mecanismo decisório por execução e não haverá réplicas concorrentes do orquestrador, protocolos de consenso como “*Raft*” ou “*Paxos*” não são necessários ao desenho experimental. Da mesma forma, os cenários de falha não incluem componentes maliciosos ou participantes que forneçam deliberadamente informações conflitantes, razão pela qual “*Byzantine Fault Tolerance*” permanece fora do escopo.

Além disso, a execução em ambiente local restringe a avaliação de mecanismos próprios de produção, como escalabilidade horizontal real, RBAC, “*sandboxing*”, recuperação distribuída e retomada persistente de “*workflows*”. Trabalhos como *KubeIntellect* incorporam esses mecanismos em ambientes “*Kubernetes*”; no presente estudo, eles são utilizados como referência para delimitar capacidades não avaliadas experimentalmente (ARDEBILI; BARTOLINI, 2025).

A reprodutibilidade externa também dependerá da disponibilidade futura do código, configurações, imagens, versões de dependências, modelo utilizado e características do “*hardware*”. O registro desses elementos favorece a reconstrução do estudo, mas não garante resultados numericamente idênticos em equipamentos ou versões de “*runtime*” diferentes. Assim, os resultados devem ser interpretados no contexto da configuração experimental documentada.

O mecanismo de *fallback* utilizado no experimento representa uma rota lógica alternativa dentro do Serviço de Estoque, e não redundância física de serviço ou réplica independente. Consequentemente, sua capacidade de recuperação não deve ser generalizada para cenários de falha total do microserviço. Essa escolha preserva o recorte de dois serviços e permite avaliar mudanças de trajetória sob condições controladas, mas não constitui avaliação de alta disponibilidade por redundância de instâncias.

A utilização do “*Celery*” como mecanismo de execução assíncrona também não constitui objeto de comparação; seus parâmetros serão comuns às duas abordagens e mantidos fixos durante a coleta.

Por fim, as limitações apresentadas não invalidam o estudo, mas delimitam seu alcance. O objetivo deste trabalho não é demonstrar superioridade universal de agentes LLM, nem validar uma arquitetura produtiva completa, mas avaliar experimentalmente indícios de desempenho, adaptabilidade e resiliência em cenários controlados. A repetição das execuções, a padronização dos arquivos de coleta, a análise estatística e a interpretação qualitativa dos “*logs*” de decisão são necessárias para reduzir conclusões baseadas em execuções isoladas e para tornar explícitas as condições sob as quais os resultados foram obtidos.

TABELA 19 – Delineamento Metodológico

Objetivos	Como atingir esse objetivo?	Onde estão esses dados?	Técnicas de coleta de dados	Técnicas de análise de dados
Objetivo Geral: Avaliar experimentalmente o uso de um agente baseado em “ <i>Large Language Model</i> ” (LLM) na tomada de decisão para orquestração dinâmica de tarefas assíncronas em uma arquitetura simulada de microserviços (NEWMAN, 2015; RICHARDSON, 2019; YU et al., 2025; ZHANG et al., 2024; CORNACCHIA et al., 2025).	Executar as duas abordagens sob a mesma topologia, o mesmo “ <i>dataset</i> ”, os mesmos parâmetros de carga e falha e o mesmo protocolo de “ <i>reset</i> ”, “ <i>readiness</i> ”, coleta e validação da execução, utilizando arquivos de configuração versionados e N_{rep} repetições válidas por combinação cenário × abordagem, conforme Subseção 4.4.1.	“ <i>Logs</i> ” de execução dos microserviços, “ <i>logs</i> ” do orquestrador, métricas dos “ <i>containers</i> ”, métricas das filas “ <i>RabbitMQ</i> ”, tempos de resposta, registros de falhas e decisões produzidas durante cada execução experimental no ambiente “ <i>Docker</i> ”.	Coleta automatizada por “ <i>logs</i> ” estruturados, “ <i>Docker Stats</i> ”, “ <i>Prometheus</i> ” e métricas do “ <i>RabbitMQ</i> ”, registrando identificador da execução, cenário, abordagem, “ <i>timestamps</i> ”, “ <i>status</i> ” da tarefa, tamanho da fila, consumo de recursos e decisões do orquestrador.	Comparação estatística entre as duas abordagens por cenário experimental, utilizando média, mediana, desvio padrão, latência P95, taxa de erro, “ <i>throughput</i> ”, tempo de recuperação, consumo de CPU/RAM, tamanho da fila, tempo de decisão do orquestrador e, na abordagem baseada em LLM, tempo de inferência do modelo.
Continua na próxima página. . .				

TABELA 19 – Continuação

Objetivos	Como atingir esse objetivo?	Onde estão esses dados?	Técnicas de coleta de dados	Técnicas de análise de dados
OE1: Implementar uma arquitetura simulada de microserviços com comunicação assíncrona, utilizando ferramentas de containerização (NEWMAN, 2015; RICHARDSON, 2019; FOWLER; LEWIS, 2017).	Arquitetura simulada descrita na Seção 4.3 e na Subseção 4.3.1.	Registros de mensagens, estados e métricas conforme Subseção 4.3.10.	Instrumentos de coleta apresentados na Subseção 4.5.3.	Análise comparativa da comunicação assíncrona conforme Subseção 4.5.4.
OE2: Desenvolver e integrar um agente LLM responsável por apoiar decisões de orquestração em cenários pré-definidos (YU et al., 2025; ARDEBILI; BARTOLINI, 2025; CORNACCHIA et al., 2025; AMAYUELAS et al., 2025).	Motor de decisão baseado em LLM implementado conforme o fluxo definido nas subseções Subseção 4.3.2 a Subseção 4.3.9.	<states.jsonl>, <decisions.jsonl>, <task_events.jsonl>, métricas do “ <i>RabbitMQ</i> ” e metadados de execução; para decisões LLM, <decisions.jsonl> contém tempo de decisão, tempo de inferência e, quando disponível, “tokens” processados, conforme Subseção 4.3.10.	Registro estruturado definido na Subseção 4.3.2 e na Subseção 4.5.3.	Análise do comportamento decisório e da validade das ações, do tempo de decisão, do tempo acumulado de inferência, da quantidade de chamadas e dos efeitos produzidos sobre o sistema, conforme Subseção 4.5.4.
Continua na próxima página. . .				

TABELA 19 – Continuação

Objetivos	Como atingir esse objetivo?	Onde estão esses dados?	Técnicas de coleta de dados	Técnicas de análise de dados
OE3: Definir métricas de avaliação para comparar o desempenho da orquestração tradicional com a baseada em LLM (CORNACCHIA et al., 2025; PEI et al., 2025; ARDEBILI; BARTOLINI, 2025; ZHANG et al., 2024).	Métricas de desempenho, resiliência e custo decisório definidas na Seção 4.5: latência média e P95, “ <i>throughput</i> ”, taxa de erro, tempo de recuperação, CPU, memória, tamanho da fila, tempo de decisão do orquestrador, tempo de inferência LLM, quantidade de decisões/chamadas e overhead decisório; “ <i>tokens</i> ” de entrada e saída como métrica auxiliar.	Arquivos de métricas gerados durante os experimentos, conforme Subseção 4.5.3.	Coleta automatizada descrita na Subseção 4.5.3.	Estatística descritiva, diferenças absolutas, intervalos de confiança, testes comparativos e medidas de tamanho de efeito por cenário, conforme Subseção 4.5.4; overhead decisório analisado separadamente do impacto sistêmico.
Continua na próxima página. . .				

TABELA 19 – Continuação

Objetivos	Como atingir esse objetivo?	Onde estão esses dados?	Técnicas de coleta de dados	Técnicas de análise de dados
OE4: Realizar experimentos controlados em diferentes cenários, incluindo execução normal, sobrecarga, falha intermitente, “ <i>timeout</i> ”, dados inconsistentes e recuperação pós-falha, coletando resultados comparáveis entre a orquestração baseada em regras e a orquestração apoiada por agente LLM (CORNACCHIA et al., 2025; ZHANG et al., 2024; PEI et al., 2025; YU; DING; SATO, 2025).	Cenários experimentais executados conforme a Seção 4.4, utilizando o protocolo de repetibilidade definido na Subseção 4.4.1, com restauração do estado inicial, configuração versionada, controle de “ <i>seeds</i> ”, “ <i>readiness</i> ”, N_{rep} repetições válidas e registro de execuções invalidadas.	Dados brutos por execução conforme a Subseção 4.3.10, parâmetros e metadados em <code><execution_metadata.json></code> , configuração em <code><experiment_config.yml></code> e registro de execuções inválidas, quando houver.	Coleta automatizada descrita na Subseção 4.5.3, associada ao protocolo de “ <i>reset</i> ”, “ <i>readiness</i> ”, versionamento e validação das execuções definido na Subseção 4.4.1.	Comparação entre abordagens conforme Subseção 4.5.4, considerando as limitações apresentadas na Seção 4.6.
Continua na próxima página. . .				

TABELA 19 – Continuação

Objetivos	Como atingir esse objetivo?	Onde estão esses dados?	Técnicas de coleta de dados	Técnicas de análise de dados
OE5: Analisar e discutir os resultados obtidos, destacando vantagens, limitações e possíveis aplicações futuras (CORNACCHIA et al., 2025; PEI et al., 2025; AMAYUELAS et al., 2025; YU; DING; SATO, 2025).	Interpretação conjunta dos ganhos de adaptabilidade/resiliência e do custo necessário para obtê-los, identificando cenários em que o overhead da camada LLM é compensado por melhoria operacional e cenários em que a linha de base baseada em regras apresenta melhor relação entre desempenho e custo, conforme Subseção 4.5.4.	Dados consolidados a partir dos registros descritos na Subseção 4.3.10 e na Subseção 4.5.3.	Organização e padronização dos dados descritas na Subseção 4.5.4.	Discussão crítica considerando as limitações apresentadas na Seção 4.6.

FONTE: Elaborada pelo autor com apoio da ferramenta Chat GPT 5.5, com base em Newman (2015), Richardson (2019), Ardebili e Bartolini (2025), Yu et al. (2025), Yu, Ding e Sato (2025), Pei et al. (2025), Cornacchia et al. (2025), Antonio GIL (2022), Creswell e Creswell (2021).

REFERÊNCIAS

- AMAYUELAS, A.; YANG, J.; AGASHE, S.; NAGARAJAN, A.; ANTONIADES, A.; WANG, X. E.; WANG, W. Self-resource allocation in multi-agent llm systems. **ArXiv**, p. 1–15, 2025. Disponível em: <<https://arxiv.org/pdf/2504.02051>>. Acesso em: 27-fev.-2026. Citado 10 vezes nas páginas 18, 34, 36, 38, 43, 44, 81, 93, 99 e 102.
- ANTONIO GIL, C. Como elaborar projetos de pesquisa. **Atlas**, v. 3, p. 17–158, 2022. Disponível em: <https://sgcd.fc.unesp.br/Home/helber-freitas/gil_como_elaborar_projetos_de_pesquisa_-anto.pdf>. Acesso em: 20-maio-2026. Citado 7 vezes nas páginas 47, 48, 80, 90, 92, 93 e 102.
- ARDEBILI, M. S.; BARTOLINI, A. Kubeintellect: A modular llm-orchestrated agent framework for end-to-end kubernetes management. **ArXiv**, p. 1–20, 2025. Disponível em: <<https://arxiv.org/pdf/2509.02449>>. Acesso em: 27-fev.-2026. Citado 15 vezes nas páginas 17, 31, 34, 35, 37, 45, 55, 87, 90, 94, 96, 97, 99, 100 e 102.
- BLACKWELL, R. E.; BARRY, J.; COHN, A. G. **Towards Reproducible LLM Evaluation: Quantifying Uncertainty in LLM Benchmark Scores**. 2025. Disponível em: <<https://arxiv.org/abs/2410.03492>>. Citado na página 40.
- CHEN, L.; ZAHARIA, M. A.; ZOU, J. Y. How is chatgpt’s behavior changing over time? **ArXiv**, abs/2307.09009, 2023. Citado na página 42.
- CNPQ, P. **Política de Integridade na Atividade Científica do CNPq**. 2026. Disponível em: <https://memoria2.cnpq.br/web/guest/view/-/journal_content/56_INSTANCE_0oED/10157/23142775>. Acesso em: 25-maio-2026. Citado 2 vezes nas páginas 44 e 49.
- CORNACCHIA, A.; ALABDULAAL, I.; SAGHIER, I.; MIRDAD, A.; FAYOUMI, O.; CANINI, M. Between promise and pain: The reality of automating failure analysis in microservices with llms. **Proceedings of the 16th ACM SIGOPS Asia-Pacific Workshop on Systems**, p. 155–167, 2025. Disponível em: <<https://dl.acm.org/doi/10.1145/3725783.3764388>>. Acesso em: 27-fev.-2026. Citado 26 vezes nas páginas 10, 11, 18, 24, 30, 35, 41, 42, 43, 44, 57, 65, 74, 80, 81, 87, 90, 92, 93, 94, 95, 98, 99, 100, 101 e 102.
- CRESWELL, J. W.; CRESWELL, J. D. **Projeto de pesquisa-: Métodos qualitativo, quantitativo e misto**. [S.l.]: Penso Editora, 2021. v. 3. 17–158 p. ISBN 852240724x. Acesso em: 20-maio-2026. Citado 6 vezes nas páginas 48, 80, 90, 92, 93 e 102.
- DAI, F.; YANG, J.; MO, Q.; ZHOU, H.; QI, L. Modeling interactions among microservices communicating via fifo or bag buffers. p. 505–518, 2021. Citado na página 22.
- DÉFAGO, X.; SCHIPER, A.; URBÁN, P. Total order broadcast and multicast algorithms: Taxonomy and survey. **ACM Comput. Surv.**, v. 36, p. 372–421, 2003. Citado na página 22.
- DONGRE, V.; ROSSI, R. A.; LAI, V. D.; YOON, D. S.; HAKKANI-TUR, D.; BUI, T. Drift no more? context equilibria in multi-turn llm interactions. **ArXiv**, abs/2510.07777, 2025. Citado na página 42.
- FENG, K.; LUO, L.; XIA, Y.; LUO, B.; HE, X.; LI, K.; ZHA, Z.; XU, B.; PENG, K. Optimizing microservice deployment in edge computing with large language models: Integrating retrieval

augmented generation and chain of thought techniques. **Symmetry**, MDPI, v. 16, n. 11, p. 1470, 2024. Disponível em: <<https://www.mdpi.com/2073-8994/16/11/1470>>. Acesso em: 27-fev.-2026. Citado 3 vezes nas páginas 35, 39 e 40.

FIDGE, C. Logical time in distributed computing systems. **Computer**, v. 24, p. 28–33, 1991. Citado na página 24.

FOWLER, M.; LEWIS, J. **Microservices**: A definition of this new architectural term. Martin Fowler, 2017. Disponível em: <<https://martinfowler.com/articles/microservices.html#CharacteristicsOfAMicroserviceArchitecture>>. Acesso em: 04-mai.-2026. Citado 7 vezes nas páginas 20, 21, 26, 27, 47, 53 e 99.

GIUSTO, C. D.; FERRE, D. A.; LAVERSA, L.; LOZES, É. A partial order view of message-passing communication models. **Proceedings of the ACM on Programming Languages**, v. 7, p. 1601 – 1627, 2022. Citado na página 22.

HE, J.; TREUDE, C.; LO, D. Llm-based multi-agent systems for software engineering: Literature review, vision, and the road ahead. **ACM Transactions on Software Engineering and Methodology**, ACM New York, NY, v. 34, n. 5, p. 1–30, 2025. Disponível em: <<https://arxiv.org/pdf/2404.04834>>. Acesso em: 21-mar.-2026. Citado 2 vezes nas páginas 32 e 34.

JIN, P.; ZHANG, S.; MA, M.; LI, H.; KANG, Y.; LI, L.; LIU, Y.; QIAO, B.; ZHANG, C.; ZHAO, P.; HE, S.; SARRO, F.; DANG, Y.; RAJMOHAN, S.; LIN, Q.; ZHANG, D. **Assess and Summarize: Improve Outage Understanding with Large Language Models**. 2023. Disponível em: <<https://arxiv.org/abs/2305.18084>>. Citado 2 vezes nas páginas 15 e 17.

KOSTIKOVA, A.; WANG, Z.; BAJRI, D.; PÜTZ, O.; PAASSEN, B.; EGER, S. **LLMs: A Data-Driven Survey of Evolving Research on Limitations of Large Language Models**. 2025. Disponível em: <<https://arxiv.org/abs/2505.19240>>. Citado na página 13.

LAMPORT, L. Time, clocks, and the ordering of events in a distributed system. **Commun. ACM**, v. 21, p. 558–565, 1978. Citado na página 24.

LAMPORT, L.; SHOSTAK, R.; PEASE, M. The byzantine generals problem. **ACM Transactions on Programming Languages and Systems (TOPLAS)**, v. 4, p. 382 – 401, 1982. Citado na página 24.

LIANG, W.; ZHANG, Y.; WU, Z.; LEPP, H.; JI, W.; ZHAO, X.; CAO, H.; LIU, S.; HE, S.; HUANG, Z.; YANG, D.; POTTS, C.; MANNING, C. D.; ZOU, J. Y. **Mapping the Increasing Use of LLMs in Scientific Papers**. 2024. Disponível em: <<https://arxiv.org/abs/2404.01268>>. Citado na página 13.

LIU, J.; WANG, K.; CHEN, Y.; PENG, X.; CHEN, Z.; ZHANG, L.; LOU, Y. **Large Language Model-Based Agents for Software Engineering: A Survey**. 2024. Disponível em: <<https://arxiv.org/abs/2409.02977>>. Citado na página 13.

LU, J.; LIU, A.; DONG, F.; GU, F.; GAMA, J.; ZHANG, G. Learning under concept drift: A review. **IEEE Transactions on Knowledge and Data Engineering**, v. 31, p. 2346–2363, 2019. Citado na página 42.

LUO, M.; SHI, X.; CAI, C.; ZHANG, T.; WONG, J.; WANG, Y.; WANG, C.; HUANG, Y.; CHEN, Z.; GONZALEZ, J.; STOICA, I. Autellix: An efficient serving engine for llm agents as general programs. **ArXiv**, abs/2502.13965, 2025. Citado 3 vezes nas páginas 18, 43 e 81.

MIAO, X.; OLIARO, G.; ZHANG, Z.; CHENG, X.; JIN, H.; CHEN, T.; JIA, Z. Towards efficient generative large language model serving: A survey from algorithms to systems. **ACM Computing Surveys**, v. 58, p. 1 – 37, 2023. Citado 3 vezes nas páginas 18, 43 e 81.

NEWMAN, S. **Building microservices: designing fine-grained systems**. 1. ed. [S.l.]: "O'Reilly Media, Inc.", 2015. 616 p. Acesso em: 04-mai.-2026. Citado 12 vezes nas páginas 10, 20, 21, 22, 23, 27, 47, 53, 94, 98, 99 e 102.

NIU, C.; ZHANG, W.; ZHAO, Y.; CHEN, Y. Energy efficient or exhaustive? benchmarking power consumption of llm inference engines. **ACM SIGENERGY Energy Informatics Review**, v. 5, p. 56 – 62, 2025. Citado na página 96.

ONGARO, D.; OUSTERHOUT, J. In search of an understandable consensus algorithm. p. 305–319, 2014. Citado na página 24.

PEI, C.; WANG, Z.; LIU, F.; LI, Z.; LIU, Y.; HE, X.; KANG, R.; ZHANG, T.; CHEN, J.; LI, J. et al. Flow-of-action: Sop enhanced llm-based multi-agent system for root cause analysis. **Companion Proceedings of the ACM on Web Conference 2025**, p. 422–431, 2025. Disponível em: <<https://arxiv.org/pdf/2502.08224>>. Acesso em: 27-fev.-2026. Citado 19 vezes nas páginas 17, 32, 34, 35, 36, 37, 41, 42, 43, 44, 57, 87, 90, 92, 93, 95, 100, 101 e 102.

PUTRA, J. A. **Cost-Aware Logging: Measuring the Financial Impact of Excessive Log Retention in Small-Scale Cloud Deployments**. 2026. Disponível em: <<https://arxiv.org/abs/2601.11584>>. Citado na página 15.

RATH, A. Agent drift: Quantifying behavioral degradation in multi-agent llm systems over extended interactions. **ArXiv**, abs/2601.04170, 2026. Citado na página 42.

RICHARDSON, C. **Microservices patterns: with examples in Java**. [S.l.]: Manning Publications, 2019. 520 p. Acesso em: 04-mai.-2026. Citado 13 vezes nas páginas 10, 20, 21, 22, 23, 25, 26, 28, 53, 94, 98, 99 e 102.

SAXENA, D.; GUPTA, I.; SINGH, A. K.; LEE, C.-N. **A Fault Tolerant Elastic Resource Management Framework Towards High Availability of Cloud Services**. 2022. Disponível em: <<https://arxiv.org/abs/2212.03547>>. Citado na página 15.

SAXENA, D.; SINGH, A. K. **A High Availability Management Model based on VM Significance Ranking and Resource Estimation for Cloud Applications**. 2022. Disponível em: <<https://arxiv.org/abs/2211.16117>>. Citado na página 15.

SHAH, J. K. Agentic ai for autonomous micro-frontend user interfaces and microservices evolution in cloud platforms. **Journal of Computer Science and Technology Studies**, p. 1195–1206, 2025. ISSN 2709-104X. Disponível em: <<https://al-kindipublisher.com/index.php/jcsts/article/view/10721/9466>>. Acesso em: 21-mar.-2026. Citado 4 vezes nas páginas 11, 23, 34 e 39.

SOLDANI, J.; BROGI, A. **Anomaly Detection and Failure Root Cause Analysis in (Micro)Service-Based Cloud Applications: A Survey**. 2021. Disponível em: <<https://arxiv.org/abs/2105.12378>>. Citado 2 vezes nas páginas 12 e 13.

SUN, W.; WANG, T.; TIAN, X.; LAN, W.; FENG, X.; LI, H.; WANG, F. **MetaKube: An Experience-Aware LLM Framework for Kubernetes Failure Diagnosis**. 2026. Disponível em: <<https://arxiv.org/abs/2603.23580>>. Citado na página 17.

WANG, Y.; LI, G.; WANG, Z.; KANG, Y.; ZHOU, Y.; ZHANG, H.; GAO, F.; SUN, J.; YANG, L.; LEE, P.; XU, Z.; ZHAO, P.; QIAO, B.; LI, L.; ZHANG, X.; LIN, Q. **Fast Outage Analysis of Large-scale Production Clouds with Service Correlation Mining**. 2021. Disponível em: <<https://arxiv.org/abs/2103.03649>>. Citado na página 15.

XIONG, Y.; WANG, J.; LI, B.; ZHU, Y.; ZHAO, Y. Self-organizing agent network for llm-based workflow automation. **ArXiv**, p. 1–16, 2025. Disponível em: <<https://arxiv.org/pdf/2508.13732>>. Acesso em: 27-fev.-2026. Citado na página 26.

YANG, Y.; CHAI, H.; SHAO, S.; SONG, Y.; QI, S.; RUI, R.; ZHANG, W. Agentnet: Decentralized evolutionary coordination for llm-based multi-agent systems. **Advances in Neural Information Processing Systems**, p. 1–16, 2026. Disponível em: <<https://arxiv.org/pdf/2504.00587>>. Acesso em: 27-fev.-2026. Citado 4 vezes nas páginas 31, 33, 40 e 43.

YU, F.; YANG, F.; QIN, X.; ZHANG, Z.; ZHANG, J.; LIN, Q.; ZHANG, H.; DANG, Y.; RAJMOHAN, S.; ZHANG, D. et al. Enabling autonomic microservice management through self-learning agents. **ArXiv**, p. 1–15, 2025. Disponível em: <<https://arxiv.org/pdf/2501.19056>>. Acesso em: 27-fev.-2026. Citado 10 vezes nas páginas 10, 28, 29, 31, 43, 55, 80, 98, 99 e 102.

YU, J.; DING, Y.; SATO, H. Dyntaskmas: A dynamic task graph-driven framework for asynchronous and parallel llm-based multi-agent systems. **Proceedings of the International Conference on Automated Planning and Scheduling**, p. 288–296, 2025. Disponível em: <<https://ojs.aaai.org/index.php/ICAPS/article/view/36130/38284>>. Acesso em: 21-mar.-2026. Citado 21 vezes nas páginas 10, 11, 17, 23, 28, 29, 30, 32, 33, 34, 35, 36, 38, 43, 80, 87, 90, 92, 93, 101 e 102.

ZHANG, L.; JIA, T.; JIA, M.; WU, Y.; LIU, A.; YANG, Y.; WU, Z.; HU, X.; YU, P. S.; LI, Y. **A Survey of AIOps in the Era of Large Language Models**. 2025. Disponível em: <<https://arxiv.org/abs/2507.12472>>. Citado na página 13.

ZHANG, S.; BAO, Y.; HUANG, S. **EDT: Improving Large Language Models' Generation by Entropy-based Dynamic Temperature Sampling**. 2024. Disponível em: <<https://arxiv.org/abs/2403.14541>>. Citado 2 vezes nas páginas 39 e 40.

ZHANG, Z.; YANG, F.; QIN, X.; ZHANG, J.; LIN, Q.; CHENG, G.; ZHANG, D.; RAJMOHAN, S.; ZHANG, Q. The vision of autonomic computing: Can llms make it a reality? **ArXiv**, p. 1–22, 2024. Disponível em: <<https://arxiv.org/pdf/2407.14402>>. Acesso em: 27-fev.-2026. Citado 18 vezes nas páginas 10, 11, 28, 29, 30, 31, 32, 34, 35, 46, 74, 80, 87, 92, 95, 98, 100 e 101.

ZHOU, X.; HUANG, H.; ZHANG, H.; HUANG, X.; SHAO, D.; ZHONG, C. **A Cross-Company Ethnographic Study on Software Teams for DevOps and Microservices: Organization, Benefits, and Issues**. 2022. Disponível em: <<https://arxiv.org/abs/2205.01446>>. Citado na página 13.

ZIFTCI, C.; LIU, R.; GREENE, S.; DALLORO, L. **LLM-Based Automated Diagnosis Of Integration Test Failures At Google**. 2026. Disponível em: <<https://arxiv.org/abs/2604.12108>>. Citado na página 17.